Politechnika Wrocławska
Wydział Informatyki i Telekomunikacji

Kierunek: Informatyczne Systemy Automatyki (ISA)
Specjalność: Inteligentne Systemy Przemysłu 4.0 (IPS)

PRACA DYPLOMOWA INŻYNIERSKA

Aplikacja wspomagająca harmonogramowanie produkcji dla wybranego zagadnienia szeregowania zadań

Bartłomiej Adam Kuk

Opiekun pracy
dr inż. Jarosław Rudy

Słowa kluczowe: harmonogramowanie, produkcja, aplikacja, optymalizacja

WROCŁAW (2025)

Streszczenie

Praca przedstawia projekt i implementację aplikacji wspomagającej harmonogramowanie produkcji dla rozszerzonego hybrydowego problemu przepływu zadań (Hybrid Flow Shop Problem), uwzględniającego czasy przezbrojeń zależnych od kolejności (SDST) oraz efekt uczenia, skracający czas operacji. Czynniki te zwiększają realizm modelu, lecz podnoszą także jego złożoność, co uzasadnia wykorzystanie metod heurystycznych i metaheurystycznych.

W pracy opisano formalny model HFSP, ograniczenia technologiczne oraz funkcję celu minimalizującą czas zakończenia produkcji (C_{max}). Wybrane metody optymalizacyjne obejmują algorytm dokładny Branch and Bound (B&B), heurystykę Minimum Setup Time First (MinSTF) oraz metaheurystykę Tabu Search. Dla każdej opracowano implementację dostosowaną do problemu z SDST i efektem uczenia.

System składa się z modułu obliczeniowego w Pythonie oraz aplikacji desktopowej JavaFX umożliwiającej definiowanie danych, wybór algorytmu i wizualizację wyników. Komunikacja odbywa się przez pliki JSON, a harmonogram prezentowany jest również jako wykres Gantta.

W eksperymentach potwierdzono, że B&B znajduje rozwiązania optymalne, lecz jest skalowalnie nieefektywny. MinSTF działa bardzo szybko, natomiast Tabu Search osiąga wysoką jakość rozwiązań przy znacznie niższym czasie obliczeń. Opracowane narzędzie stanowi efektywne wsparcie procesu planowania produkcji.

Abstract

This thesis presents the design and implementation of an application supporting production scheduling for an extended Hybrid Flow Shop Problem (HFSP) with Sequence-Dependent Setup Times (SDST) and a learning effect that reduces processing times. These factors increase model realism but also computational complexity, motivating the use of heuristic and metaheuristic methods.

The thesis defines the HFSP model, its constraints, and the objective of minimizing the makespan (C_{max}). Three optimization approaches were implemented: the exact Branch and Bound (B&B), the heuristic Minimum Setup Time First (MinSTF), and the Tabu Search metaheuristic.

The system consists of a Python-based optimization engine and a JavaFX desktop application for data input, algorithm selection, and result visualization. JSON is used for data exchange, and results are additionally shown as Gantt charts.

Experimental results show that B&B yields optimal solutions but scales poorly, MinSTF is very fast, and Tabu Search provides high-quality schedules with significantly lower computation time. The developed application effectively supports decision-making in production scheduling.

Spis treści

1 Wstęp **1**
1.1 Cel pracy . . . 1
1.2 Zakres pracy . . . 2
1.3 Struktura pracy . . . 2

2 Przegląd literatury **3**
2.1 Integrating Sequence-Dependent Setup Times and Blocking in Hybrid Flow Shop Scheduling to Minimize Total Tardiness — Kurt (2025) . . . 3
2.2 Hybrid Flow Shop Scheduling through Reinforcement Learning — Pugliese, Romano, Lin, Zhu, Tan (2025) . . . 3
2.3 A Real Time Human Machine Logistics Collaborative Scheduling Method Considering Workers' Learning and Forgetting Effects — Yang, Li, Luo, Li, Wen (2025) . . . 4
2.4 MRLM: A meta-reinforcement learning-based metaheuristic for hybrid flow-shop scheduling problem with learning and forgetting effects — Zhang, Shao, Shao, Chen, Pi (2024) . . . 4

3 Sformułowanie problemu **7**
3.1 Opis problemu szeregowania zadań . . . 7
3.2 Metody optymalizacji problemu HFS . . . 8
3.2.1 Algorytm dokładny – Branch and Bound (B&B) . . . 9
3.2.2 Algorytm heurystyczny – Minimum Setup Time First . . . 10
3.2.3 Algorytm metaheurystyczny – Tabu Search . . . 11
3.3 Założenia aplikacji . . . 13

4 Projekt rozwiązania **15**
4.1 Architektura systemu . . . 15
4.1.1 Model architektury . . . 15
4.1.2 Wybrane technologie . . . 16
4.2 Opis algorytmów . . . 17
4.2.1 Algorytm dokładny – Branch and Bound (B&B) . . . 17
4.2.2 Algorytm metaheurystyczny – Tabu Search . . . 18
4.3 Struktura danych wejściowych i wyników optymalizacji . . . 19
4.4 Projekt Graficznego Interfejsu Użytkownika . . . 22

5 Implementacja systemu **25**

5.1 Implementacja modułu obliczeniowego i wizualizacyjnego — Python 25
5.1.1 Środowisko i biblioteki 25
5.1.2 Organizacja kodu i modularyzacja 26
5.1.3 Reprezentacja danych dla problemu 27
5.1.4 Implementacja Algorytmów 36
5.1.5 Uruchomienie skryptu Pythona 47
5.1.6 Moduł wizualizacji harmonogramu 51
5.2 Implementacja warstwy aplikacji klienckiej — Java 55
5.2.1 Interfejs użytkownika oraz wprowadzenie danych lub ich generacja . . . 55
5.2.2 Integracja z procesem Pythonem 60
5.2.3 Wizualizacja wyników 62
5.3 Zarządzanie zależnościami i procedura uruchomieniowa 64

6 Weryfikacja poprawności i napotkane problemy 65
6.1 Metodyka testowania i weryfikacja funkcjonalna 65
6.2 Eksperymenty obliczeniowe i ocena wydajności 67
6.3 Analiza napotkanych problemów 71

7 Zakończenie 73

1. Wstęp

W ostatnich latach sektor przemysłowy podlega dynamicznym zmianom wynikającym z cyfryzacji, automatyzacji oraz rosnącej zmienności rynkowej. Najnowsze analizy globalne wskazują, że przedsiębiorstwa produkcyjne mierzą się z presją skracania czasów realizacji zamówień, zwiększania elastyczności zasobów oraz redukcji kosztów operacyjnych, co podkreślono m.in. w *World Manufacturing Report 2024* [7]. Transformacja cyfrowa staje się fundamentem współczesnych strategii rozwojowych i prowadzi do rosnącej potrzeby stosowania zaawansowanych narzędzi analitycznych wspierających podejmowanie decyzji.

W praktyce przemysłowej szczególne znaczenie zyskuje koncepcja *Przemysłu 4.0*, obejmująca integrację systemów cyberfizycznych, automatyzację, Internet Rzeczy oraz analizę danych. Zgodnie z perspektywami przedstawionymi w raporcie *World Manufacturing Report 2024*, technologie te stają się kluczowym czynnikiem zwiększającym konkurencyjność przedsiębiorstw oraz efektywność procesów wytwórczych [7]. Dynamiczny rozwój systemów produkcyjnych powoduje, że zagadnienia harmonogramowania stają się jednym z najważniejszych obszarów badań operacyjnych, szczególnie w kontekście rosnącej złożoności współczesnych linii produkcyjnych.

Jednym z kluczowych problemów opisujących realne środowisko produkcyjne jest Hybrydowy przepływowy problem szeregowania zadań (Hybrid Flow Shop Problem, HFSP), stanowiący uogólnienie klasycznych modeli przepływu pracy. Problem ten uwzględnia wieloetapowość produkcji oraz możliwość równoległego wykonywania operacji, co czyni go szczególnie przydatnym do modelowania rzeczywistych procesów przemysłowych.

W praktycznych zastosowaniach problem HFSP często podlega rozszerzeniu o czynniki zwiększające realizm modelu. Jednym z nich jest *efekt uczenia się*, odzwierciedlający skracanie czasu wykonywania operacji wraz z rosnącą liczbą przetwarzanych zadań. Drugim istotnym elementem są *czasy przezbrojeń zależnych od kolejności* (Sequence-Dependent Setup Times), wymagające uwzględnienia dodatkowych opóźnień wynikających ze zmiany konfiguracji maszyn. Połączenie tych czynników prowadzi do dalszego zwiększenia złożoności HFSP, co uzasadnia stosowanie metod heurystycznych i metaheurystycznych.

1.1. Cel pracy

Głównym celem niniejszej pracy jest zaprojektowanie, implementacja i weryfikacja funkcjonalności aplikacji wspomagającej harmonogramowanie produkcji dla wybranego zagadnienia szeregowania zadań: hybrydowego przepływowego problemu szeregowania zadań z czasami przezbrojeń zależnymi od kolejności i efektem uczenia się.

Realizacja tego celu wymaga stworzenia kompleksowego środowiska informatycznego, które umożliwi użytkownikowi definiowanie, rozwiązywanie oraz wizualizację harmonogramu dla zadanego kryterium minimalizacji czasu całkowitego zakończenia produkcji.

1.2. Zakres pracy

Zakres niniejszej pracy obejmuje kompleksową realizację aplikacji desktopowej, której głównym przeznaczeniem jest wsparcie w rozwiązywaniu złożonego zagadnienia szeregowania zadań: *Hybrydowego Przepływowego Problemu Szeregowania Zadań* z uwzględnieniem czasów przezbrojeń zależnych od kolejności i efektu uczenia się. Realizacja projektu obejmowała następujące kluczowe funkcjonalności i implementacje:

1. *Modelowanie i Definicja Problemu:* Stworzenie mechanizmów umożliwiających intuicyjne definiowanie parametrów instancji problemowej, w tym liczby etapów, konfiguracji maszyn równoległych oraz wczytywania danych wejściowych z plików `*.JSON`.

2. *Implementacja Algorytmów Optymalizacyjnych:* Opracowanie i wdrożenie różnorodnych metod rozwiązywania problemu, obejmujących zarówno algorytmy heurystyczne, jak i metaheurystyczne, w celu znalezienia optymalnego harmonogramu minimalizującego czas realizacji wszystkich zadań – *makespan* ($C_{\max}$).

3. *Wizualizacja i Prezentacja Wyników:* Zapewnienie funkcjonalności graficznej prezentacji otrzymanego harmonogramu w postaci czytelnego *Diagramu Gantta* , z możliwością wizualnego rozróżnienia czasu przetwarzania od czasu przezbrojenia, w celu późniejszej analizy.

4. *Zarządzanie Wynikami i Instancjami:* Wdrożenie opcji eksportu i importu danych, umożliwiające trwały zapis zarówno definicji problemu, jak i szczegółowych wyników optymalizacji.

5. *Weryfikacja Wydajności:* Przeprowadzenie szczegółowych eksperymentów obliczeniowych mających na celu ocenę efektywności i złożoności czasowej zaimplementowanych algorytmów dla różnych rozmiarów instancji.

1.3. Struktura pracy

Praca została podzielona na 7 rozdziałów. W rozdziale 2 omówiono przegląd literatury, który skupia się na współczesnych rozwiązaniach hybrydowego przepływowego problemu szeregowania zadań. W rozdziale 3 zaprezentowane jest sformułowanie problemu, w którym przedstawiono dokładne założenia problemu, metody jego rozwiązania oraz założenia aplikacji. W rozdziale 4 zaprezentowano projektowanie rozwiązania. Przedstawia on konkretne rozwiązania technologiczne, wykorzystane parametry algorytmów optymalizacji oraz projekt interfejsu użytkownika. Następnie w rozdziale 5 przedstawiono implementację całości systemu, a w rozdziale 6 przedstawiono weryfikację poprawności, wyniki badań oraz napotkane problemy. W ostatnim rozdziale 7 przedstawiono wnioski oraz potencjalne możliwości rozwoju.

2. Przegląd literatury

2.1. Integrating Sequence-Dependent Setup Times and Blocking in Hybrid Flow Shop Scheduling to Minimize Total Tardiness — Kurt (2025)

Autor podejmuje problem harmonogramowania w środowisku Hybrid Flow Shop, w którym uwzględnia dwa istotne aspekty wpływające na realizację zadań: czasy przezbrojenia zależne od kolejności operacji (sequence-dependent setup times) oraz brak buforów pomiędzy kolejnymi etapami produkcji (blocking). Głównym celem optymalizacji jest minimalizacja całkowitego opóźnienia zadań (total tardiness), co stanowi alternatywę dla najczęściej spotykanego kryterium minimalizacji czasu zakończenia (makespan).

W pracy przedstawiono dwa modele oparte na programowaniu całkowitoliczbowym (MILP, *Mixed-Integer Linear Programming*), które umożliwiają rozwiązywanie małych instancji problemu. Dla większych instancji zaproponowano podejścia heurystyczne: algorytm lokalnego przeszukiwania (ILS, *Iterated Local Search*) oraz hybrydowy algorytm genetyczny (HGA,*Hybrid Genetic Algorithm*). Wyniki eksperymentów wykazały, że metody heurystyczne pozwalają na uzyskanie dobrej jakości rozwiązań w krótkim czasie obliczeniowym.

Mimo że autor nie uwzględnia w modelu efektu uczenia się, praca dostarcza wartościowych narzędzi i wniosków w kontekście integracji przezbrojeń i blokowania w harmonogramowaniu produkcji typu HFS. Stanowi punkt odniesienia dla projektowania rozwiązań metaheurystycznych w bardziej złożonych modelach, uwzględniających dodatkowe zjawiska dynamiczne. [3]

2.2. Hybrid Flow Shop Scheduling through Reinforcement Learning — Pugliese, Romano, Lin, Zhu, Tan (2025)

Autorzy przedstawiają systematyczny przegląd literatury dotyczący zastosowania metod uczenia przez wzmacnianie (RL, *Reinforcement Learning*) w problemach harmonogramowania typu Hybrid Flow Shop, ze szczególnym uwzględnieniem rozszerzeń obejmujących efekt uczenia się, zapominania oraz czasy przezbrojeń. W pracy dokonano klasyfikacji istniejących badań pod względem stosowanych algorytmów RL (m.in. Q-learning, PPO – *Proximal Policy Optimization*, DDPG – *Deep Deterministic Policy Gradient*), reprezentacji stanów, funkcji nagrody oraz strategii eksploracji.

Przegląd pokazuje, że większość istniejących prac koncentruje się na uproszczonych wersjach HFS, natomiast modele zawierające jednocześnie efekty uczenia, zapominania i setup times są rzadkością. Autorzy identyfikują lukę badawczą w postaci braku kompleksowych podejść integrujących te trzy zjawiska w jednym środowisku RL. Wskazano również na rosnącą

skuteczność podejść hybrydowych, łączących RL z metaheurystykami, takimi jak algorytmy genetyczne czy symulowane wyżarzanie.

Wnioski zawarte w pracy stanowią cenne źródło wiedzy dla dalszych badań nad RL w kontekście rzeczywistych systemów produkcyjnych, w których istotna jest adaptacja do dynamicznych warunków oraz zdolność do modelowania efektów zależnych od historii operacji. [5]

2.3. A Real Time Human Machine Logistics Collaborative Scheduling Method Considering Workers' Learning and Forgetting Effects — Yang, Li, Luo, Li, Wen (2025)

Autorzy proponują innowacyjne podejście do harmonogramowania w czasie rzeczywistym w inteligentnych warsztatach, które integruje w jednym modelu zasoby ludzkie, maszynowe i logistyczne. Kluczowym elementem jest dynamiczny model efektywności pracowników, oparty na efektach uczenia się i zapominania, pozwalający na logistyczno-produkcyjne grupowanie zasobów oraz adaptacyjne dopasowanie zadań do dostępnych grup. Badania eksperymentalne potwierdzają, że rozważenie tych efektów prowadzi do znaczącej redukcji czasu zakończenia zadań, opóźnień i zużycia energii.

Metodologia obejmuje model matematyczny opisujący efekty uczenia i zapominania, algorytm dopasowujący zadania do grup zasobów na podstawie luzu czasowego oraz mechanizm samoorganizacji zespołów w czasie rzeczywistym. Praca stanowi wartościowe odniesienie przy badaniu metod opartych na uczeniu ze wzmocnieniem lub metaheurystykach w kontekście harmonogramowania z efektami uczenia i zapominania w zmiennym środowisku produkcyjnym. [8]

2.4. MRLM: A meta-reinforcement learning-based metaheuristic for hybrid flow-shop scheduling problem with learning and forgetting effects — Zhang, Shao, Shao, Chen, Pi (2024)

Autorzy proponują innowacyjną metodę opartą na meta-uczeniu ze wzmocnieniem (*meta-reinforcement learning*), która służy do rozwiązywania rozszerzonego Hybrydowego Problemu Przepływu Zadań, z uwzględnieniem efektu uczenia się i zapominania. Artykuł przedstawia model MRLM (*Meta-Reinforcement Learning-based Metaheuristic*), który adaptuje się do różnych instancji problemu oraz dynamicznych zmian w systemie produkcyjnym. W modelu zastosowano architekturę *Actor-Critic* oraz technikę task-specific embeddingów.

W eksperymentach porównawczych z klasycznymi metaheurystykami (np. GA – *Genetic Algorithm*, SA – *Simulated Annealing*, TS – *Tabu Search*) zaproponowana metoda wykazuje

lepszą zdolność generalizacji oraz krótszy czas znajdowania wysokiej jakości rozwiązań harmonogramu. Wyniki sugerują, że podejście oparte na meta-uczeniu może skutecznie wspierać harmonogramowanie w zmiennych środowiskach przemysłowych. [10]

Podsumowanie przeglądu literatury

Przegląd literatury dotyczy nowoczesnych metod rozwiązywania problemu harmonogramowania w środowisku Hybrid Flow Shop, ze szczególnym uwzględnieniem czasów przezbrojeń, efektów uczenia się i zapominania oraz zastosowań sztucznej inteligencji, w tym uczenia przez wzmacnianie i metaheurystyk.

Literatura wskazuje, że integracja SDST, efektu uczenia i zapominania pozostaje otwartym zagadnieniem badawczym. *Kurt (2025)* analizuje HFS z przezbrojeniami zależnymi od kolejności i blokowaniem, proponując modele MILP oraz heurystyki (ILS, HGA). *Pugliese i in. (2025)* dokonują przeglądu metod RL, wskazując brak podejść uwzględniających jednocześnie uczenie, zapominanie i przezbrojenia. *Yang i in. (2025)* modelują dynamiczną efektywność pracowników w czasie rzeczywistym, a *Zhang i in. (2024)* proponują zaawansowaną metaheurystykę MRLM opartą na meta-RL, świadcząc o rosnącym znaczeniu sztucznej inteligencji w harmonogramowaniu.

Wnioski: Omówione prace ukazują *ewolucję podejść* do problemu harmonogramowania HFS: od klasycznych modeli optymalizacyjnych, przez zastosowanie heurystyk, aż po inteligentne systemy oparte na uczeniu maszynowym. Integracja *efektów uczenia się, zapominania i przezbrojeń* nadal stanowi wyzwanie badawcze, jednak nowoczesne rozwiązania hybrydowe oparte na RL i metaheurystykach wskazują obiecujący kierunek dalszych badań.

3. Sformułowanie problemu

3.1. Opis problemu szeregowania zadań

Rozważany jest problem szeregowania zadań w środowisku produkcyjnym typu *hybrydowy przepływowy* (Hybrid Flow Shop – HFS), w którym występuje s etapów (faz), i każdy etap $a \in \{1, \ldots, s\}$ dysponuje określoną liczbą m_a maszyn pracujących równolegle, gdzie $m_a \geq 1$, a istnieje przynajmniej jeden etap a taki, że $m_a \geq 2$. Celem jest zaplanowanie przetwarzania n zadań na dostępnych zasobach w taki sposób, aby uwzględnić ograniczenia technologiczne oraz zminimalizować wartość przyjętego kryterium jakości harmonogramu.

Każde zadanie $J_j \in \{J_1, \ldots, J_n\}$ składa się z s operacji $O_{j1}, O_{j2}, \ldots, O_{js}$, które muszą zostać wykonane kolejno, tj. operacja O_{ja} musi zakończyć się przed rozpoczęciem $O_{j(a+1)}$. Operacja O_{ja} może zostać przypisana do jednej z dostępnych maszyn na etapie a, przy czym wybór maszyny oraz kolejność operacji wpływa na ostateczny harmonogram.

Elementy modelu

- **Liczba maszyn:** Każdy etap $a \in \{1, \ldots, s\}$ dysponuje m_a maszynami, które pracują równolegle i mogą niezależnie wykonywać operacje przypisane do tego etapu. Zbiór liczby maszyn na wszystkich etapach oznaczamy jako $M = \{m_1, m_2, \ldots, m_s\}$ i traktujemy jako konfigurację zasobów produkcyjnych systemu.

- **Czasy przetwarzania:** Czas bazowy realizacji operacji O_{ja} przez maszynę k oznaczany jako $p_{jak} > 0$.

- **Przezbrojenia zależne od kolejności (SDST):** Po wykonaniu operacji O_{ja} przypisanej do maszyny k, przejście do kolejnej operacji $O_{j'a}$ może wymagać przezbrojenia. Czas przezbrojenia zależy od pary kolejnych zadań i oznaczany jest jako $s_{jj'}^{ak}$.

- **Efekt uczenia się:** Zakłada się, że czas realizacji operacji skraca się w miarę wykonywania kolejnych zadań dzięki akumulacji doświadczenia. Zmodyfikowany czas przetwarzania uwzględniający efekt uczenia opisuje wzór:

  $$p_{jak}^{(r)} = p_{jak} \cdot r^{-\alpha} \tag{3.1}$$

  gdzie:

  - r – liczba operacji już zrealizowanych na maszynie k przed bieżącą operacją,
  - $\alpha > 0$ – współczynnik uczenia (większa wartość oznacza silniejszy efekt),
  - p_{jak} – bazowy czas operacji bez efektu uczenia.

  Efekt ten kumuluje się lokalnie – osobno na każdej maszynie – i jest niezależny od typu realizowanego zadania.

Założenia i ograniczenia

1. Każde zadanie musi przejść przez wszystkie etapy w ustalonej kolejności.
2. Każda maszyna może jednocześnie przetwarzać tylko jedną operację.
3. Każda operacja może być wykonana tylko raz i tylko przez jedną maszynę danego etapu.
4. Przezbrojenia są wymagane między kolejnymi operacjami na tej samej maszynie.
5. Efekt uczenia się działa lokalnie dla każdej maszyny – niezależnie od typu zadania – i nie zanika w czasie.

Funkcja celu

Celem optymalizacji jest minimalizacja czasu zakończenia realizacji ostatniego zadania w harmonogramie, określanego jako *makespan*. Formalnie:

$$C_{\max} = \max_{j \in \{1,\ldots,n\}} C_j \tag{3.2}$$

gdzie C_j to czas zakończenia ostatniej operacji zadania j. W związku z tym funkcja celu przyjmuje postać:

$$\min C_{\max} \tag{3.3}$$

Minimalizacja wartości $C_{\max}$ pozwala na skrócenie całkowitego czasu realizacji wszystkich zadań w systemie.

3.2. Metody optymalizacji problemu HFS

W celu rozwiązania problemu harmonogramowania zadań w środowisku Hybrid Flow Shop z przezbrojeniami zależnymi od kolejności oraz efektem uczenia, konieczne było zastosowanie odpowiednio dobranych algorytmów optymalizacyjnych. Ze względu na złożoność obliczeniową zagadnienia, obejmującą cechy problemów klasy NP-trudnych, w projekcie wykorzystano zarówno podejścia heurystyczne, jak i metaheurystyczne.

W niniejszej sekcji przedstawiono przegląd zastosowanych metod, podzielonych na trzy główne kategorie:

- **Algorytmy dokładne** – wykorzystujące deterministyczne podejście do poszukiwania rozwiązania optymalnego.
- **Algorytmy heurystyczne** – szybkie metody konstrukcyjne, generujące rozwiązania początkowe na podstawie lokalnych reguł priorytetów.
- **Algorytmy metaheurystyczne** – strategie inspirowane zjawiskami naturalnymi, umożliwiające eksplorację przestrzeni rozwiązań w sposób probabilistyczny.

3.2.1. Algorytm dokładny – Branch and Bound (B&B)

Algorytm *Branch and Bound* (B&B) jest metodą dokładną wykorzystywaną do znajdowania rozwiązania optymalnego poprzez systematyczne przeszukiwanie przestrzeni wszystkich możliwych permutacji zadań. W kontekście problemu *Hybrid Flow Shop* (HFS) z przezbrojeniami zależnymi od kolejności oraz efektem uczenia się, metoda ta pozwala uwzględnić dodatkowe ograniczenia technologiczne podczas wyznaczania harmonogramu minimalizującego czas zakończenia rezliacji zadań $C_{\max}$.

W najgorszym przypadku złożoność obliczeniowa algorytmu wynosi $O(n!)$, ponieważ rozważane są wszystkie możliwe kolejności zadań. W praktyce jednak stosowane są mechanizmy ograniczające (*bounds*), które umożliwiają odrzucanie (*pruning*) nieperspektywicznych gałęzi drzewa przeszukiwań, co pozwala znacząco zmniejszyć liczbę analizowanych rozwiązań w porównaniu do przeglądu zupełnego.

Algorytm opiera się na dwóch podstawowych typach ograniczeń:

1. **Dolne ograniczenie (Lower Bound, LB)** – określa minimalny możliwy koszt (czas zakończenia) dla częściowego rozwiązania. Pozwala odrzucać gałęzie drzewa, które nie mogą prowadzić do lepszego rozwiązania niż aktualnie znane.

2. **Górne ograniczenie (Upper Bound, UB)** – stanowi oszacowanie maksymalnego kosztu harmonogramu dla danej częściowej permutacji, co przyspiesza odrzucanie rozwiązań gorszych od najlepszego dotychczas znalezionego.

Algorytm przyjmuje jako dane wejściowe zbiór n zadań, macierze czasów operacji p_{jak}, czasy przezbrojeń $s_{jj'}^{ak}$ oraz współczynnik uczenia α. Wynikiem działania jest optymalna permutacja π^* minimalizująca $C_{\max}$, dzięki której można wyznaczyć jej harmonogramem. Dzięki zastosowaniu ograniczeń $LB(\pi')$ i $UB(\pi')$, algorytm może efektywnie odrzucać nieoptymalne gałęzie drzewa, skracając czas działania przy zachowaniu gwarancji optymalności.

Algorithm 3.1: Branch and Bound dla HFS z przezbrojeniami i efektem uczenia

Input: Zbiór n zadań $J = \{J_1, J_2, \ldots, J_n\}$, macierze czasów operacji p_{jak}, czasy przezbrojeń $s^{ak}_{jj'}$, współczynnik uczenia α

Output: Optymalna permutacja π^* minimalizująca $C_{\max}$

```
π* ← ∅ ;                                   // najlepsza znaleziona permutacja
C* ← ∞ ;                                   // koszt najlepszego rozwiązania
Q ← kolejka z pustą permutacją π
while Q niepusta do
    π ← Q.pop()
    if |π| = n then
        if C_max(π) < C* then
            C* ← C_max(π)
            π* ← π
    else
        foreach j ∈ zadania nieobecne w π do
            π' ← π ∘ j ;                   // dodaj zadanie j na koniec permutacji
            if LB(π') < C* then
                if UB(π') < C* then
                    Q.push(π') ;           // rozwiń tylko obiecujące gałęzie
return π*
```

W algorytmie:

- π – reprezentuje częściową lub pełną permutację zadań,
- $LB(\pi')$ – dolne oszacowanie kosztu harmonogramu dla częściowej permutacji π',
- $C_{\max}(\pi)$ – całkowity czas realizacji zadań dla permutacji π,
- $UB(\pi')$ – górne oszacowanie kosztu dla danej częściowej permutacji.

Metoda Branch and Bound, mimo wysokiej złożoności obliczeniowej, stanowi punkt odniesienia dla metod przybliżonych i metaheurystycznych. W badaniach nad problemem *Hybrid Flow Shop* wykorzystywana jest często jako algorytm porównawczy do oceny jakości rozwiązań generowanych przez heurystyki i metaheurystyki [2, 4].

3.2.2. Algorytm heurystyczny – Minimum Setup Time First

Algorytm *Minimum Setup Time First* (MSTF lub MinSTF) jest prostą, lecz skuteczną heurystyką wykorzystywaną w problemach szeregowania z przezbrojeniami zależnymi od kolejności (SDST). W każdej iteracji wybierane jest kolejne zadanie, które powoduje *najmniejszy dodatkowy koszt przezbrojenia* względem ostatnio zaplanowanego zadania. Dzięki temu metoda minimalizuje lokalne koszty zmian konfiguracji maszyn, tworząc kolejność zadań w sposób szybki i intuicyjny.

Metoda MSTF nie gwarantuje globalnej optymalności rozwiązania (w przeciwieństwie do algorytmów dokładnych, takich jak metoda Branch and Bound o złożoności $O(n!)$), lecz umożliwia w czasie wielomianowym, zwykle $O(n^2)$, uzyskanie harmonogramów o akceptowalnej jakości. Z tego względu często stosowana jest jako procedura inicjalizująca dla bardziej złożonych metod metaheurystycznych.

Algorithm 3.2: Min Setup Time First dla HFS-SDST z efektem uczenia

Input: Zbiór n zadań $J = \{J_1, J_2, \ldots, J_n\}$, macierze czasów operacji p_{jak}, czasy przezbrojeń $s^{ak}_{jj'}$, współczynnik uczenia α

Output: Permutacja $\pi = (\pi_1, \pi_2, \ldots, \pi_n)$ minimalizująca $C_{\max}$

```
1  π ← ∅ ;                                                  // inicjalizacja permutacji
2  J_reszta ← J ;                                           // zadania do zaplanowania
3  J_poprzednie ← zadanie startowe (np. zerowe lub fikcyjne)
4  while J_reszta ≠ ∅ do
5      foreach J_i ∈ J_reszta do
6          Oblicz średni czas przezbrojenia s^eff_{J_poprzednie,i} z uwzględnieniem α
7      Wybierz J_i z minimalnym s^eff_{J_poprzednie,i}
8      π ← π ∘ J_i ;                                        // dodaj zadanie J_i do permutacji
9      J_poprzednie ← J_i
10     J_reszta ← J_reszta \ {J_i}
11 return π
```

Algorytm MSTF charakteryzuje się wielomianową złożonością czasową rzędu $O(n^2)$, ponieważ każda z n iteracji wymaga przeglądu co najwyżej $n - i$ pozostałych zadań.
To sprawia, że jest on znacznie szybszy niż algorytmy dokładne, które muszą przeszukać całą przestrzeń permutacji o złożoności $O(n!)$, oferując jednocześnie rozwiązanie o akceptowalnej jakości. Skuteczność tego typu podejść konstrukcyjnych została szeroko potwierdzona w literaturze badawczej [6, 9].

W kontekście Problemów Harmonogramowania typu Hybrid Flow Shop z przezbrojeniami zależnymi od kolejności oraz efektem uczenia się, algorytm MSTF jest szczególnie wartościowy. Może on efektywnie uwzględniać fakt, że czasy przezbrojeń i operacji ulegają skróceniu wraz z kolejną realizacją zadań na tej samej maszynie. Badania (np. [6, 9]) wykazały, że heurystyki konstrukcyjne bazujące na regułach lokalnej minimalizacji czasów (tzw. *MSTF-like rules*) pozwalają znacząco skrócić czas obliczeń, zachowując przy tym wysoką jakość uzyskiwanych harmonogramów w złożonych modelach HFS z efektami uczenia i przezbrojeniami.

3.2.3. Algorytm metaheurystyczny – Tabu Search

Kompromisem pomiędzy dokładnością a czasem obliczeń są algorytmy metaheurystyczne, które umożliwiają uzyskanie rozwiązań o wysokiej jakości w rozsądnym czasie obliczeniowym. Przykładem takiego podejścia jest algorytm Tabu Search, opierający się na probabilistycznym przeszukiwaniu przestrzeni rozwiązań w celu uniknięcia lokalnych minimów oraz zwiększenia efektywności procesu optymalizacji.

Algorytm ten w sposób iteracyjny przeszukuje sąsiedztwo aktualnego rozwiązania, dążąc do znalezienia lepszego wariantu w jego otoczeniu. Aby zapobiec powracaniu do wcześniej

odwiedzonych rozwiązań i uniknąć cyklicznego przeszukiwania tych samych obszarów, stosuje się listę zabronionych ruchów – Tabu List. Długość tej listy może być określona statycznie lub dostosowywana dynamicznie w zależności od przyjętej strategii działania algorytmu. W pewnych sytuacjach możliwe jest jednak złamanie zakazu tabu – jeśli dany ruch prowadzi do rozwiązania lepszego niż dotychczas najlepsze znalezione. Takie wyjątki są dopuszczane na podstawie tzw. kryterium aspiracji, które zwiększa elastyczność algorytmu i pozwala uniknąć pominięcia potencjalnie optymalnych rozwiązań.

Definicja sąsiedztwa może przyjmować różne formy — od prostych operacji zamiany dwóch zadań w harmonogramie po bardziej złożone manipulacje, takie jak obrót lub przesunięcie podciągu permutacji. Pozwala to na eksplorację różnych regionów przestrzeni rozwiązań i zwiększa szansę na znalezienie rozwiązania bliskiego optymalnemu.

Złożoność obliczeniowa algorytmu zależy od wielu czynników, takich jak liczba iteracji, długość listy tabu, sposób definiowania sąsiedztwa oraz przede wszystkim liczba zadań w harmonogramie. Pomimo wpływu tych parametrów, algorytm Tabu Search zachowuje złożoność wielomianową, co sprawia, że jest istotnie szybszy od metod dokładnych, przy jednoczesnym zachowaniu wysokiej jakości otrzymywanych rozwiązań.

Algorithm 3.3: Klasyczny algorytm Tabu Search dla problemu HFS z przezbrojeniami i efektem uczenia

Input: Zbiór n zadań, macierze czasów operacji p_{jak}, czasy przezbrojeń $s^{ak}_{jj'}$, współczynnik uczenia α

Output: Permutacja $\pi = (\pi_1, \pi_2, \ldots, \pi_n)$ minimalizująca $C_{\max}$

```
π ← permutacja początkowa (np. wygenerowana heurystycznie);
π* ← π ;                                   // najlepsze dotąd znalezione rozwiązanie
tabuList ← ∅;
for iter = 1 to maxIter do
    sąsiedzi ← zbiór rozwiązań powstałych w wyniku zastosowania wybranej operacji
      sąsiedztwa do π;
    wybierz π' z sąsiedztwa o najmniejszym C_max, nieuwzględnione w tabuList lub
      spełniające kryterium aspiracji;
    if C_max(π') < C_max(π*) then
        π* ← π';
    zaktualizuj tabuList (dodaj wykonany ruch, usuń najstarsze wpisy);
    π ← π';
return π*
```

Algorytm Tabu Search jest jedną z najczęściej stosowanych metaheurystyk w problemach harmonogramowania o dużej złożoności obliczeniowej. Dzięki wykorzystaniu pamięci krótkoterminowej (listy tabu) oraz mechanizmów kontrolnych, takich jak kryterium aspiracji, umożliwia efektywne przeszukiwanie przestrzeni rozwiązań i unikanie lokalnych minimów. Jego elastyczność pozwala na dostosowanie struktury sąsiedztwa, długości listy tabu oraz kryteriów zatrzymania do specyfiki danego problemu. W literaturze algorytm ten wykazał wysoką skuteczność w rozwiązywaniu problemów typu Hybrid Flow Shop z dodatkowymi ograniczeniami, takimi jak czasy przezbrojeń, brak przestojów czy efekt uczenia się, przewyższając często metody dokładne pod względem relacji jakość–czas obliczeń [1].

Podsumowanie metod optymalizacji

W rozdziale przedstawiono różne podejścia do rozwiązywania problemu harmonogramowania w środowisku Hybrid Flow Shop z przezbrojeniami zależnymi od kolejności oraz efektem uczenia. Omówione metody obejmowały:

- algorytm dokładny (Branch and Bound), gwarantujący rozwiązanie optymalne, lecz nieefektywny obliczeniowo dla dużych instancji,
- algorytmy heurystyczne (np. Min Setup Time First), które zapewniają szybkie, choć niekoniecznie optymalne harmonogramy
- metaheurystyki (np. Tabu Search), stanowiące kompromis między jakością rozwiązania a czasem jego wyznaczania.

Każde z tych podejść znajduje zastosowanie zależnie od rozmiaru instancji, dostępnego czasu obliczeniowego oraz potrzeby co do jakości harmonogramu.

3.3. Założenia aplikacji

Projektowana aplikacja desktopowa ma stanowić narzędzie wspomagające proces optymalizacji harmonogramu produkcji. Jej głównym celem jest umożliwienie użytkownikowi łatwego definiowania danych wejściowych, uruchamiania algorytmu optymalizacyjnego oraz prezentacji uzyskanego harmonogramu w czytelnej formie graficznej.

Wymagania funkcjonalne

Aplikacja powinna umożliwiać:

1. wczytywanie danych wejściowych z pliku (np. w formacie JSON),
2. ręczne definiowanie parametrów zadania i instancji problemu,
3. wybór algorytmu optymalizacyjnego: aplikacja powinna umożliwiać uruchomienie co najmniej jednego algorytmu z każdego z trzech podejść:
   (a) Podejście heurystyczne
   (b) Podejście dokładne
   (c) Podejście metaheurystyczne
4. uruchomienie algorytmu i wyświetlenie wyników na ekranie,
5. wizualizację harmonogramu w formie wykresu Gantta,
6. eksport wyników do pliku (np. w formacie JSON lub jako obraz PNG).

Wymagania niefunkcjonalne

Aplikacja powinna spełniać następujące wymagania pozafunkcjonalne:

1. zapewniać czytelny i intuicyjny interfejs użytkownika,
2. gwarantować krótkie czasy odpowiedzi dla standardowych rozmiarów danych,
3. działać w trybie offline, bez konieczności dostępu do Internetu.

4. Projekt rozwiązania

4.1. Architektura systemu

Architektura systemu została zaplanowana tak, aby zapewnić modularność, czytelność struktury kodu oraz możliwość przyszłej rozbudowy. System będzie składał się z dwóch głównych komponentów: części obliczeniowej (Python) oraz interfejsu użytkownika (JavaFX).

4.1.1. Model architektury

System zostanie zaimplementowany w architekturze warstwowej (ang. layered architecture), składającej się z następujących głównych warstw:

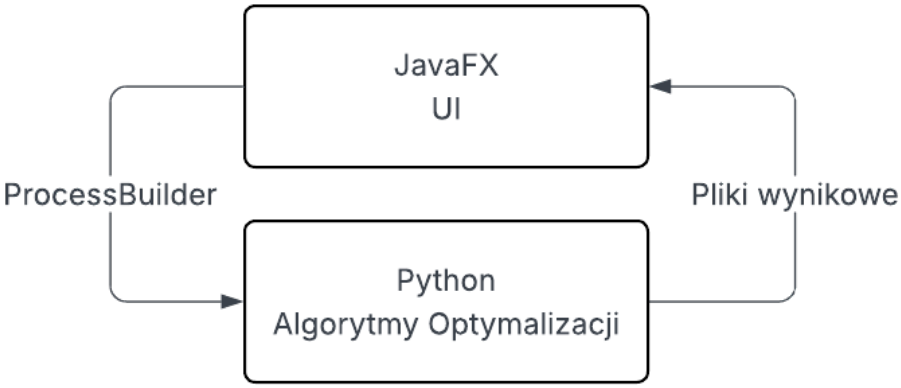
- **Warstwa prezentacji (UI)** Zaimplementowana w technologii JavaFX. Odpowiada za interakcję z użytkownikiem, w tym wczytanie danych, wybór algorytmu, konfigurację parametrów oraz prezentacje wyników.
- **Warstwa komunikacji (uruchamianie procesu)** Odpowiada za przekazywanie danych między interfejsem użytkownika a częścią obliczeniową. Wykorzystuje mechanizm `ProcessBuilder` do uruchamiania skryptów Pythona bezpośrednio z poziomu aplikacji Java, przekazując dane w formacie JSON oraz odczytując wynik z nowego pliku JSON z gotowym harmonogramem.
- **Warstwa logiki dziedzinowej (silnik optymalizacji)** Zaimplementowana w Pythonie. Odpowiada za uruchamianie algorytmów optymalizacyjnych, przetwarzanie danych wejściowych i generowanie harmonogramu. Każdy algorytm stanowi osobny moduł, co pozwala na łatwe porównywanie i testowanie.

4.1.2. Wybrane technologie

- **Część obliczeniowa (silnik):** Python 3.13.4
- **Interfejs graficzny (UI):** JavaFX 21.10.1
- **Komunikacja między warstwami:** `ProcessBuilder` (uruchamianie Pythona z poziomu Javy)
- **Wizualizacja:** Wykres Gantta generowany w Pythonie i wyświetlany w JavaFX jako gotowy obraz
- **Zapis/odczyt danych:** Pliki JSON (dla danych wejściowych/wyjściowych)

Diagram komunikacji między komponentami

Diagram poniżej przedstawia uproszczony przepływ danych i komunikację pomiędzy głównymi komponentami systemu.



Rysunek 4.1: Diagram komunikacji pomiędzy JavaFX a modułem optymalizacji w Pythonie

4.2. Opis algorytmów

W tej części pracy przedstawiono szczegółowy opis zastosowanych algorytmów optymalizacyjnych oraz ich elementów składowych, wykorzystanych w procesie rozwiązywania problemu harmonogramowania w środowisku typu *Hybrid Flow Shop*. Opis obejmuje konkretne rozwiązania implementacyjne, takie jak sposób wyznaczania dolnych i górnych ograniczeń (LB i UB), przyjęte reguły sąsiedztwa, kryteria aspiracji, strukturę listy tabu oraz funkcje oceny harmonogramów. W odróżnieniu od części teoretycznej, niniejszy rozdział koncentruje się na praktycznej realizacji i parametryzacji poszczególnych algorytmów, uwzględniającej specyfikę analizowanego problemu.

4.2.1. Algorytm dokładny – Branch and Bound (B&B)

Dolne ograniczenie (LB)

W klasycznych algorytmach typu Branch and Bound stosowane są dolne ograniczenia wyprowadzane na podstawie długości ścieżki krytycznej [2] lub równomiernego obciążenia maszyn z minimalnymi czasami przezbrojeń [4]. W prezentowanej pracy, ze względu na specyfikę rozszerzonego modelu HFSP (uczenie się oraz SDST), przyjęto praktyczne oszacowanie dolnego ograniczenia jako czas zakończenia częściowego harmonogramu:

$$\mathrm{LB}(\pi') = C_{max}(\pi'),$$

Takie podejście zapewnia, że dolne ograniczenie jest zawsze *poprawne* - nie przecenia możliwości osiągnięcia lepszego rozwiązania poprzez rozszerzenie π' o pozostałe zadania, co gwarantuje znalezienie rozwiązania optymalnego przez algorytm Branch & Bound.

Górne ograniczenie (UB)

Górne ograniczenie UB_{best} reprezentuje wartość C_{max} dla najlepszego dotychczas znalezionego rozwiązania pełnego π. Nie jest ono liczone w każdym węźle drzewa, lecz aktualizowane tylko po uzyskaniu kompletnego harmonogramu (pełnej permutacji zadań):

$$\mathrm{UB}_{best} = \min(\mathrm{UB}_{best}, C_{max}(\pi)) \tag{4.1}$$

Dzięki temu górne ograniczenie odzwierciedla rzeczywiste wyniki obliczone dla pełnych rozwiązań, co znacząco redukuje koszt obliczeniowy algorytmu. Węzły są odcinane wyłącznie na podstawie relacji między wartościami dolnego ograniczenia a aktualnym najlepszym rozwiązaniem:

$$\mathrm{LB}(\pi') \geq \mathrm{UB}_{best} \Rightarrow \text{gałąź zostaje odcięta.} \tag{4.2}$$

W implementacji przyjęto, że początkowa wartość górnego ograniczenia UB_{best} ustalana jest na podstawie harmonogramu uzyskanego dla *permutacji naturalnej*, czyli kolejności zadań zgodnej z ich indeksem.

Algorytm działa do momentu pełnego przeszukania przestrzeni rozwiązań i zawsze zwraca *rozwiązanie optymalne*. Gałęzie są odcinane wyłącznie na podstawie relacji między wartościami $\mathrm{LB}(\pi')$ i UB_{best}.

4.2.2. Algorytm metaheurystyczny – Tabu Search

Rozwiązanie początkowe

Jako rozwiązanie początkowe zastosowano heurystykę *Minimum Setup Time First (MinSTF)*, która konstruuje sekwencję poprzez iteracyjne dobieranie kolejnych zadań minimalizujących czas przezbrojenia:

$$J_{\text{next}} = \arg\min_{J_j \notin \pi} s_{i,j},$$

gdzie $s_{i,j}$ oznacza czas przezbrojenia przy przejściu z zadania J_i do J_j. Heurystyka ta generuje spójną sekwencję startową S_0, skracającą przejścia między zadaniami, co ma istotne znaczenie w problemie SDST.

Struktura sąsiedztwa

Zbiór sąsiedztwa $N(S)$ definiowany jest za pomocą pojedynczego typu ruchu:

- **Swap** – zamiana miejscami dwóch zadań J_i oraz J_j w bieżącej permutacji.

W każdej iteracji generowany jest zbiór sąsiadów poprzez wykonanie wszystkich możliwych ruchów typu *swap*, a następnie wybierany jest sąsiad S', który minimalizuje funkcję celu $f(S)$.

Lista tabu

W celu uniknięcia cyklicznego odwiedzania tych samych konfiguracji zastosowano *listę tabu (TL)* przechowującą informacje o ostatnich wykonanych ruchach. Długość listy ustalono eksperymentalnie na wartość:

$$L_{tabu} = 10. \tag{4.3}$$

Po wykonaniu ruchu, para zamienionych zadań $((J_i, J_j)$ jaki i $((J_j, J_i))$ trafia na listę tabu i pozostaje na niej przez L_{tabu} kolejnych iteracji.

Kryterium aspiracji

Ruch oznaczony jako tabu może zostać zaakceptowany, jeśli prowadzi do rozwiązania lepszego niż dotychczasowe najlepsze:

$$f(S_{new}) < f(S_{best}). \tag{4.4}$$

Dzięki temu możliwe jest wyjście z lokalnego minimum i poprawa globalnej jakości rozwiązania.

Parametry i przebieg działania algorytmu

- **Funkcja celu:** minimalizacja całkowitego czasu zakończenia ostatniego zadania w harmonogramie.
- **Heurystyka początkowa:** *MinSTF (Minimum Setup Time First).*

- **Długość listy tabu:** $L_{tabu} = 10$ — lista przechowywana w formie słownika, co umożliwia sprawdzanie przynależności ruchu w czasie stałym $O(1)$.
- **Liczba iteracji:** $k_{max} = 100$.
- **Kryterium zatrzymania:** brak poprawy przez $\delta = 10$ kolejnych iteracji.

Algorytm rozpoczyna działanie od rozwiązania S_0 wygenerowanego metodą *MinSTF*, a następnie iteracyjnie przeszukuje przestrzeń sąsiedztwa poprzez ruchy typu *swap*. Po każdej iteracji aktualizowana jest lista tabu oraz najlepsze znalezione rozwiązanie S_{best}. Po spełnieniu kryterium zatrzymania algorytm zwraca harmonogram o najmniejszej wartości $C_{\max}$.

Dobór parametrów poszczególnych algorytmów oparto na analizie dostępnych badań oraz artykułów naukowych stanowiących podstawę opracowanego rozwiązania (omówionych w sekcji 3.2).

4.3. Struktura danych wejściowych i wyników optymalizacji

Na podstawie opisanych wcześniej metod optymalizacji (zob. sekcja 3.2) oraz schematu komunikacji (zob. rysunek 4.1) pomiędzy interfejsem graficznym a modułem obliczeniowym w Pythonie, w niniejszej części zaprezentowano strukturę danych wejściowych oraz oczekiwany format wyników generowanych przez silnik optymalizacji.

Dane wejściowe

Dane wejściowe przekazywane do silnika optymalizacji zapisywane są w formacie `*.JSON`, co zapewnia łatwą integrację z interfejsem graficznym oraz niewielki narzut pamięciowy. Struktura pliku zawiera wszystkie niezbędne informacje do uruchomienia procesu harmonogramowania.

1. **Typ algorytmu** – nazwa metody optymalizacji, która ma zostać użyta (np. `bnb`, `greedy`, `tabu`).
   Typ: ciąg znaków (string).
2. **Liczba etapów** – liczba etapów produkcji oznaczona w pracy jako s.
   Typ: liczba całkowita (int).
3. **Liczba zadań** – liczba zadań do zaplanowania, oznaczona jako n lub $J = \{J_1, J_2, \ldots, J_n\}$.
   Typ: liczba całkowita (int).
4. **Liczba maszyn w każdym etapie** – liczność zbioru maszyn w etapach $a = 1, \ldots, s$, oznaczona jako m_a (łącznie $M = \{m_1, m_2, \ldots, m_s\}$).
   Typ: tablica liczb całkowitych (array of int).
5. **Współczynnik uczenia się** – oznaczony w pracy jako α, określający tempo skracania czasu operacji w zależności od liczby powtórzeń.
   Typ: liczba zmiennoprzecinkowa (float).

6. **Etapy uczące się** – binarny ciąg długości s, w którym 1 oznacza, że dany etap podlega efektowi uczenia się (np. `"0010"` oznacza, że tylko trzeci etap zawiera etap uczący się).
   Typ: ciąg binarny (string).

7. **Czasy operacji** p_{jak} – trójwymiarowa tablica reprezentująca podstawowe czasy przetwarzania zadania j w etapie a na maszynie k.
   Typ: tensor o wymiarach $n \times s \times m_a$.

8. **Czasy przezbrojeń** $s_{jj'}^{ak}$ – czasy przezbrojeń zależne od kolejności zadań j oraz j' na maszynie k w etapie a. Wartości te są przechowywane jako czterowymiarowa macierz.
   Typ: tensor o wymiarach $n \times n \times s \times m_a$.

Dzięki takiej strukturze danych możliwe jest elastyczne definiowanie instancji problemu i ich automatyczne przetwarzanie niezależnie od liczby zadań, etapów i maszyn. Format `*.JSON` dodatkowo ułatwia testowanie oraz integrację z różnymi środowiskami programistycznymi.

Poniżej przedstawiono fragment przykładowego pliku wejściowego:

```
{
  "algorithm": "greedy", "num_stages": 2,
  "num_jobs": 3, "machines_per_stage": [1, 2],
  "learning_coefficient": 0.9,
  "learning_stages": "10",
  "processing_times": [
    [[10], [8, 9]], [[9], [7, 10]], [[11], [6, 8]]
  ],
  "setup_times": [
    [
      [[1], [[3, 2]]],
      [[3], [[2, 3]]],
      [[5], [[4, 5]]]
    ],
    [
      [[3], [[1, 2]]],
      [[2], [[3, 2]]],
      [[4], [[3, 2]]]
    ],
    [
      [[6], [[3, 4]]],
      [[4], [[2, 3]]],
      [[3], [[1, 1]]]
    ]
  ]
}
```

Listing 4.1: Przykładowa struktura pliku `input.json`

Zawarte w pliku (listing 4.1) dane opisują przypadek z trzema zadaniami i dwoma etapami, z jedną maszyną przypisaną do pierwszego etapu oraz dwiema maszynami do drugiego etapu. Etap pierwszy jest uczący się (oznaczenie `"learning_stages": "10"`), a czasy operacji oraz przezbrojeń są zapisane w postaci zagnieżdżonych tablic, odpowiednio:

- `processing_times[i][a][k]` – czas operacji p_{iak} dla zadania i, etapu a, maszyny k,

- `setup_times[i][j][a][k]` – czas przezbrojenia s_{ij}^{ak} pomiędzy zadaniami $i \to j$ na etapie a i maszynie k. Relacja $i \to i$ oznacza przygotowanie maszyny do produkcji zadania i, o wskazuje na konieczność przygotowania maszyny przed rozpoczęciem produkcji, czyli w sytuacji, gdy maszyna nie była wcześniej używana do żadnego zadania.

Taka struktura danych umożliwia łatwą serializację i deserializację, co ułatwia integrację pomiędzy komponentami systemu: silnikiem optymalizacyjnym napisanym w Pythonie oraz interfejsem graficznym w Javie. Format JSON zapewnia przejrzystość, rozszerzalność oraz możliwość szybkiej walidacji danych wejściowych i wyjściowych.

Wynik działania algorytmu

W wyniku działania algorytmu uzyskiwany jest harmonogram z przypisaniem operacji do maszyn, kolejnością ich realizacji oraz czasami rozpoczęcia i zakończenia, co umożliwia obliczenie C_j oraz $C_{\max}$. Struktura wyjściowa zwracana przez silnik optymalizacyjny jest reprezentowana w formacie JSON, co umożliwia łatwą integrację z interfejsem użytkownika oraz dalsze przetwarzanie danych. Plik wynikowy zawiera listę obiektów reprezentujących zaplanowane akcje na każdej maszynie (operacje i przezbrojenia), wraz z czasem ich trwania.

Każdy wpis zawiera następujące pola:

1. `type` — typ operacji: `"operation"` lub `"setup"`,
2. `job` — numer zadania (dla typu `operation`),
3. `from_job`, `to_job` — zadania źródłowe i docelowe (dla typu `setup`),
4. `stage` — numer etapu produkcji,
5. `machine` — indeks maszyny,
6. `start` — czas rozpoczęcia,
7. `end` — czas zakończenia.

Przykładowa struktura pliku `schedule.json`:

```
    { "type": "setup", "from_job": 1, "to_job": 1, "stage": 1,
        "machine": 1, "start": 0, "end": 1 },
    { "type": "operation", "job": 1, "stage": 1, "machine": 1,
        "start": 1, "end": 10 },
    { "type": "setup", "from_job": 2, "to_job": 2, "stage": 1,
        "machine": 2, "start": 0, "end": 3 },
    { "type": "operation", "job": 2, "stage": 1, "machine": 2,
        "start": 3, "end": 10 },
    { "type": "setup", "from_job": 3, "to_job": 3, "stage": 1,
        "machine": 2, "start": 10, "end": 15 },
    { "type": "operation", "job": 3, "stage": 1, "machine": 2,
        "start": 15, "end": 23 },
    { "type": "setup", "from_job": 1, "to_job": 1, "stage": 2,
        "machine": 1, "start": 0, "end": 3 },
    { "type": "operation", "job": 1, "stage": 2, "machine": 1,
```

```
16          "start": 3, "end": 13 },
17      { "type": "setup", "from_job": 2, "to_job": 2, "stage": 2,
18          "machine": 2, "start": 3, "end": 6 },
19      { "type": "operation", "job": 2, "stage": 2, "machine": 2,
20          "start": 6, "end": 16 },
21      { "type": "setup", "from_job": 3, "to_job": 3, "stage": 2,
22          "machine": 2, "start": 6, "end": 10 },
23      { "type": "operation", "job": 3, "stage": 2, "machine": 2,
24          "start": 10, "end": 18 }
25  ]
```

Listing 4.2: Przykładowy fragment harmonogramu wynikowego `schedule.json`

W pliku wynikowym może również znajdować się dodatkowy parametr globalny:

- `"time"` — łączny czas działania algorytmu optymalizacji (czas obliczeń, podany w sekundach),

Przykładowy nagłówek pliku z takim parametrem:

```
1 {
2   "time": 0.317,
3   "schedule": [
4     ...
5   ]
6 }
```

Listing 4.3: Przykładowy fragment pliku wynikowego `schedule.json`

Dodatkowo, po zakończeniu działania algorytmu generowany jest plik `gantt_chart.png`, który przedstawia wykres Gantta całego harmonogramu, z rozróżnieniem etapów, maszyn oraz zadań i przezbrojeń. Wykres ten może być automatycznie wyświetlany w GUI lub zapisywany do raportu końcowego.

4.4. Projekt Graficznego Interfejsu Użytkownika

Graficzny interfejs użytkownika (GUI, *Graphical User Interface*) aplikacji zostanie zaprojektowany z myślą o prostocie obsługi oraz intuicyjności. Jego głównym celem będzie umożliwienie użytkownikowi wygodnego wczytywania danych wejściowych, definiowania parametrów problemu, uruchamiania algorytmów optymalizacyjnych oraz prezentacji wyników w formie graficznej. Interfejs będzie zbudowany przy użyciu technologii `JavaFX`, co zapewni responsywność oraz możliwość łatwego rozszerzenia w przyszłości.

Funkcjonalności GUI

1. **Interaktywność** – GUI umożliwi użytkownikowi szybkie przejście między ekranami w celu wczytania danych wejściowych, wyboru algorytmu, uruchomienia obliczeń oraz przeglądania wyników optymalizacji.

2. **Przejrzysta prezentacja wyników** – wyniki algorytmów optymalizacyjnych będą prezentowane w czytelnej formie, zarówno liczbowo, jak i wizualnie. Użytkownik zobaczy harmonogram z dokładnymi czasami operacji na maszynach oraz wykres Gantta, ułatwiający analizę optymalnego rozkładu zadań.

3. **Możliwość eksportu wyników** – po zakończeniu procesu optymalizacji użytkownik będzie miał możliwość eksportu wyników zarówno w formacie JSON, jak i w postaci wykresu Gantta w formacie JPG.

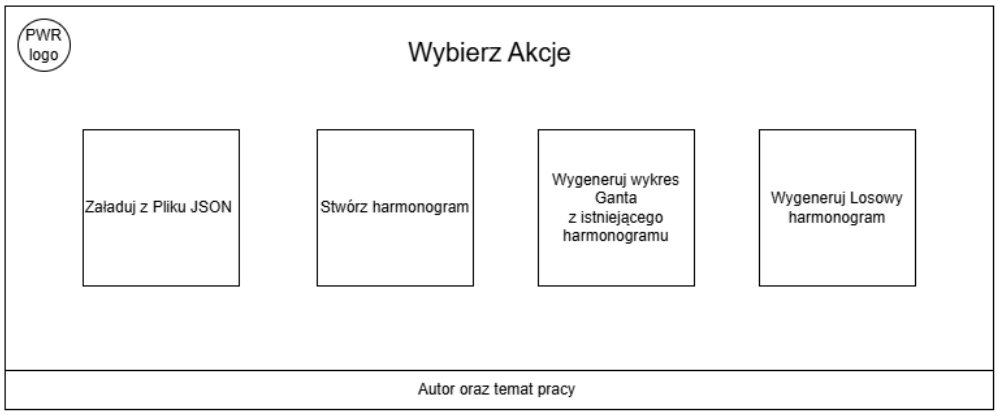
4. **Zarządzanie danymi wejściowymi** – użytkownik będzie mógł wczytywać pliki JSON zawierające dane o zadaniach, maszynach, czasach przetwarzania i przezbrojeniach. Aplikacja umożliwi również ręczne wprowadzenie danych w razie potrzeby.

GUI będzie zaprojektowane w taki sposób, aby każdy etap procesu – od załadowania danych, przez wybór algorytmu, po wizualizację wyników – był jasny i prosty do zrealizowania. Interfejs będzie maksymalnie uproszczony, umożliwiając łatwą nawigację i szybkie przejście do kluczowych funkcji aplikacji.

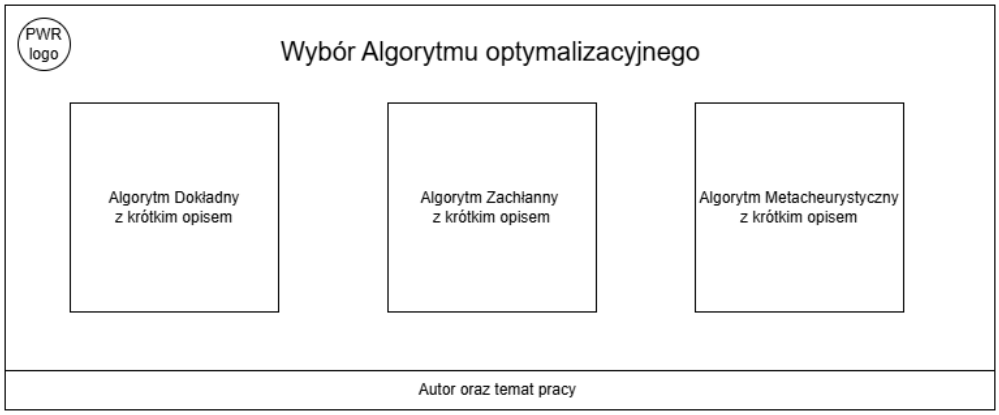
Szkielet interfejsu użytkownika

Przedstawione poniżej szkielety interfejsu (wireframe) obrazują strukturę oraz układ ekranów aplikacji. Każdy z widoków odpowiada kolejnemu etapowi interakcji użytkownika – od wyboru akcji i algorytmu, poprzez wprowadzanie danych, aż po prezentację wyników działania programu.

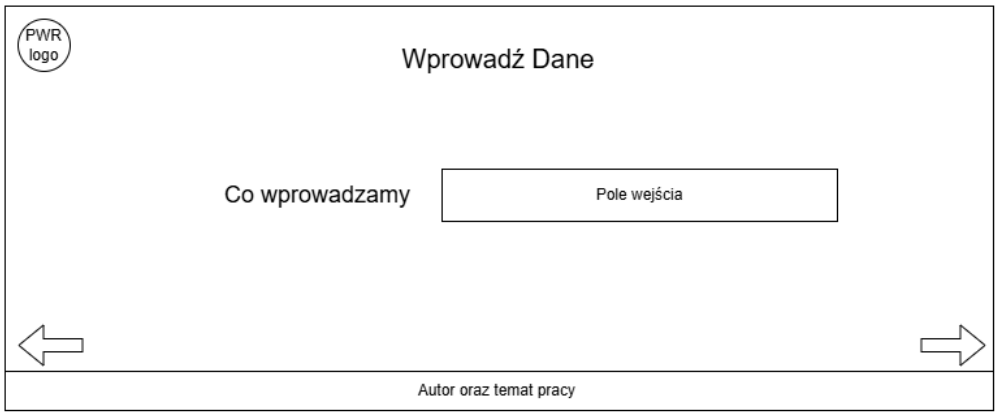
Dane opisujące instancję problemu będą wprowadzane krok po kroku w sposób sekwencyjny. Podczas wykonywania obliczeń aplikacja będzie prezentować nazwę uruchomionego algorytmu oraz szczegółową specyfikację analizowanego przypadku.



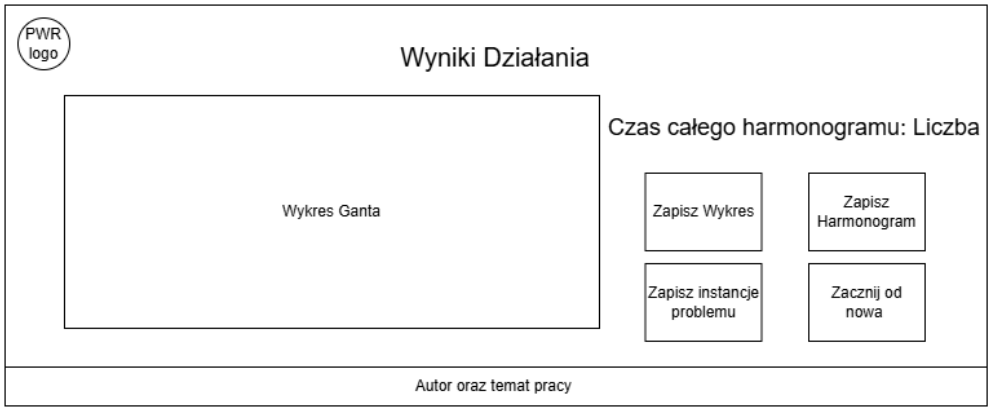
Rysunek 4.2: Widok okna startowego po uruchomieniu aplikacji.



Rysunek 4.3: Widok ekranu wyboru algorytmu optymalizacyjnego.



Rysunek 4.4: Widok ekranu wprowadzania parametrów instancji problemu.



Rysunek 4.5: Widok ekranu z wynikami działania algorytmu.

5. Implementacja systemu

Niniejszy rozdział stanowi techniczne rozwinięcie koncepcji zaprojektowanej w poprzedniej części pracy. Przedstawiono w nim szczegóły realizacji środowiska programistycznego, ze szczególnym uwzględnieniem podziału na logiczne warstwy aplikacji. Opis struktury implementacyjnej został podzielony na dwa główne bloki tematyczne, odpowiadające zastosowanym technologiom: moduł obliczeniowy zrealizowany w języku Python oraz warstwę interfejsu użytkownika i orkiestracji procesów wykonaną w technologii Java. Poniżej omówiono kluczowe algorytmy, struktury danych oraz mechanizmy integracji obu środowisk.

5.1. Implementacja modułu obliczeniowego i wizualizacyjnego — Python

Kluczowym elementem projektowanego systemu jest silnik optymalizacyjny zaimplementowany w języku *Python*. Moduł ten odpowiada za wykonywanie obliczeń, realizację algorytmów szeregowania, generowanie graficznej reprezentacji harmonogramu w formie wykresu Gantta oraz gotowego harmonogramu w formacie `*.JSON`.

5.1.1. Środowisko i biblioteki

W projekcie wykorzystano interpreter języka `Python` w wersji 3.13.4. Do realizacji modułu obliczeniowego wykorzystano zestaw bibliotek standardowych oraz pakiety zewnętrzne, których dobór podyktowany był wymaganiami wydajnościowymi oraz koniecznością generowania wizualizacji. Poniżej przedstawiono charakterystykę wykorzystanych narzędzi.

Biblioteki standardowe:

`sys` Interfejs niskiego poziomu do interpretera Pythona, zapewniający dostęp do strumieni wejścia/wyjścia oraz metadanych środowiska wykonawczego. Wykorzystywana do obsługi kodowania UTF-8 w systemie Windows poprzez rekonfigurację buforów wyjściowych oraz do kontroli zakończenia programu w przypadku błędów krytycznych.

`os` Interfejs do systemu operacyjnego, umożliwiający zarządzanie plikami tymczasowymi oraz czyszczenie zasobów po zakończeniu obliczeń. Używana do tworzenia katalogów wyników, normalizacji ścieżek i zarządzania strukturą plików projektu.

`argparse` Wysokopoziomowy parser argumentów wiersza poleceń z walidacją i automatyczną generacją dokumentacji. Umożliwia uruchamianie programu w różnych trybach (pojedyncze obliczenie, generowanie wykresów) poprzez zdefiniowane flagi (`-test`, `-plot`) i argumenty pozycyjne (ścieżka do pliku wejściowego).

`json` Moduł do serializacji i deserializacji obiektów, zapewniający ustrukturyzowany format wymiany danych między modułami systemu. Wykorzystywany do wczytywania instancji problemu oraz eksportu wyników harmonogramowania.

`traceback` Narzędzie do diagnostyki błędów, umożliwiające generowanie szczegółowych raportów o wyjątkach (ang. *stack trace*). Stosowane przy obsłudze błędów generowania diagramów Gantta i innych operacji krytycznych.

Biblioteki zewnętrzne:

`matplotlib` (3.5.0 lub nowsza) Biblioteka wizualizacyjna służąca do generowania wykresów Gantta oraz prezentacji statystyk procesu optymalizacji. Wykorzystywana do tworzenia diagramów Gantta z wizualizacją operacji i przezbrojenia.

5.1.2. Organizacja kodu i modularyzacja

Strukturę modułu obliczeniowego, zgodną ze standardem hierarchii pakietów języka Python, przedstawia Listing 5.1.

```
Python/
|-- benchmark.py
|-- main.py
|-- README.md
|-- requirements.txt
|-- algorithms/
|   |-- base_algorithm.py
|   |-- branch_and_bound.py
|   |-- greedy.py
|   |-- tabu_search.py
|   \-- __init__.py
|-- core/
|   |-- job.py
|   |-- machine.py
|   |-- schedule.py
|   \-- __init__.py
\-- utils/
    |-- gantt_chart.py
    \-- __init__.py
```

Listing 5.1: Struktura plików modułu obliczeniowego

Przedstawiony podział na pakiety odzwierciedla trójwarstwową architekturę modułu obliczeniowego, co zapewnia *wysoki stopień modularyzacji* oraz ułatwia testowanie poszczególnych komponentów systemu. Poniżej przedstawiono role kluczowych pakietów:

`core/` Pakiet ten stanowi *rdzeń logiczny* aplikacji, definiując abstrakcyjne klasy odzwierciedlające model problemu: `job.py`, `machine.py` oraz `schedule.py`. Jest to *jedyna* część kodu, która bezpośrednio implementuje definicje hybrydowego problemu przepływowego (HFS) i nie zawiera logiki optymalizacyjnej.

algorithms/ Zawiera *gotowe implementacje algorytmów optymalizacyjnych* w formie klas. Standaryzacja poprzez wspólne metody bazowe (zdefiniowane w `base_algorithm.py`) umożliwia szybką wymianę i porównywanie algorytmów bez ingerencji w moduły `core/`.

utils/ Pakiet narzędziowy, odpowiedzialny za funkcje pomocnicze, które nie są bezpośrednio związane z logiką szeregowania. Znajduje się tu moduł `gantt_chart.py` zapewniający integrację z biblioteką `matplotlib` i realizację wizualizacji.

Kluczowe pliki na poziomie głównym (`Python/`) odpowiadają za wejście/wyjście i zarządzanie: `main.py` jest głównym punktem wejścia, a `benchmark.py` służy do uruchamiania testów wydajnościowych na różnych instancjach problemu.

5.1.3. Reprezentacja danych dla problemu

Wspólnym elementem dla wszystkich algorytmów optymalizacyjnych zaimplementowanych w systemie jest klasa `Schedule`, która reprezentuje instancję problemu w kontekście optymalizacji. Klasa ta, umieszczona w pakiecie `core/`, przechowuje wszystkie parametry statyczne definiujące problem. Tworzy ona również obiekty `Job` oraz `Machine` , które posiadają odpowiednie metody, tak aby czasy operacji i przezbrojeń były zgodne z parametrami instancji problemu, zdefiniowanymi lub automatycznie wygenerowanymi.

```
class Schedule:
    def __init__(self):
        self._raw_data = None
        self.jobs = []
        self.machines = []
        self.schedule = []
        self.num_stages = 0
        self.machines_per_stage = []
        self.Cmax = 0.0
        self.learning_stages = ""
        self.learning_coeff = 0.0
```

Listing 5.2: Konstruktor klasy Schedule.

Na listingu 5.2 przedstawia konstruktor klasy `Schedule`. Metoda `__init__` inicjalizuje kluczowe atrybuty danych i stanu do wartości początkowych. Atrybuty takie jak `num_stages`, `machines_per_stage`, `learning_coeff` oraz `jobs` i `machines` stanowią definicję instancji problemu (parametry statyczne). Z kolei `schedule` (lista operacji i przezbrojeń) oraz `Cmax` (wartość funkcji celu) stanowią reprezentację rozwiązania i jego jakości. Atrybut `_raw_data` przechowuje surowe dane wejściowe z pliku JSON, co umożliwia łatwe resetowanie stanu instancji.

```python
def load_from_json(self, file_path):
    with open(file_path, "r") as f:
        data = json.load(f)
    self._raw_data = data
    self.num_stages = data["num_stages"]
    self.machines_per_stage = data["machines_per_stage"]
    self.learning_coeff = data.get("learning_coefficient", 0.0)
    self.learning_stages = data.get("learning_stages", "0" * self.num_stages)
    self.setup_times = data["setup_times"]
    self.processing_times = data["processing_times"]
    self.jobs = [
        Job(
            job_id=j,
             stages=self.processing_times[j],
             setup_times=self.setup_times
         )
         for j in range(len(self.processing_times))
    ]
    self.machines = [
         [Machine(stage_id=s, machine_id=m, is_learning=self.learning_stages[s] == "1")
          for m in range(self.machines_per_stage[s])]
         for s in range(self.num_stages)
    ]
```

Listing 5.3: Ładowanie parametrów instancji z pliku *.JSON.

Listing 5.3 przedstawia metodę `load_from_json()`, która jest odpowiedzialna za parsowanie pliku wejściowego i utworzenie pełnej instancji problemu. Metoda pobiera parametry statyczne, które podał użytkownik, i które znajdują się w pliku `*.JSON`.

W końcowej fazie metoda dynamicznie tworzy obiekty `Job` i `Machine`. Obiekty `Job` są inicjalizowane danymi o czasach operacji i macierzami przezbrojeń, natomiast obiekty `Machine` są grupowane w tablicę dwuwymiarową, reprezentującą etapy i maszyny, z odpowiednią flagą `is_learning` ustawioną w zależności od konfiguracji `learning_stages`. Dzięki temu cała instancja problemu jest hermetyzowana w obiekcie `Schedule`, gotowym do wykorzystania przez algorytmy optymalizacyjne.

Klasa `Machine`

Każdy obiekt klasy `Machine` zawiera odpowiednie metody, które umożliwiają bezproblemowe operacje, pozwalające na stworzenie harmonogramu:

```
def __init__(self, stage_id: int, machine_id: int,
                        is_learning: bool = False):
    self.stage_id = stage_id
    self.machine_id = machine_id
    self.state = "free"
    self.last_job = None
    self.is_learning = is_learning
    self.available_time = 0.0
    self.processed_jobs = 0
```

Listing 5.4: Konstruktor klasy `Machine`.

W konstruktorze klasy Machine (listing 5.4) inicjalizowane są podstawowe atrybuty opisujące stan maszyny na danym etapie procesu produkcyjnego. Parametry `stage_id` oraz `machine_id` jednoznacznie identyfikują maszynę w strukturze Hybrid Flow Shop. Atrybut state określa aktualny stan pracy maszyny i przyjmuje wartość początkową *free*.

Pole `available_time` przechowuje najwcześniejszy moment, od którego maszyna może rozpocząć kolejne zadanie, co pozwala na prawidłowe wyznaczenie chronologii operacji. Atrybut `last_job` przechowuje identyfikator ostatnio przetwarzanego zadania, co umożliwia obliczenie czasu przezbrojenia zależnego od kolejności (SDST).

Jeżeli dany etap posiada efekt uczenia się, parametr `is_learning` aktywuje mechanizm modyfikacji czasu przetwarzania w oparciu o liczbę wcześniej wykonanych zadań — wartość ta jest przechowywana w polu `processed_jobs`.

```
def start_setup(self, start_time: float,
                setup_duration: float) -> float:
     self.state = "setup"
     end_time = start_time + float(setup_duration)
     self.available_time = end_time
     return end_time
```

Listing 5.5: Rozpoczęcie przezbrojeń danego zadania.

```
def start_processing(self, job_id: int, start_time: float,
                process_duration: float) -> float:
    self.state = "busy"
    end_time = start_time + float(process_duration)
    self.available_time = end_time
    self.last_job = job_id
    self.processed_jobs += 1
    return end_time
```

Listing 5.6: Rozpoczęcie przetwarzania danego zadania.

```
def finish_job(self):
        self.state = "free"
```

Listing 5.7: Zwolnienie statusu maszyny.

Metody przedstawione w listingach 5.5, 5.6 oraz 5.7 reprezentują podstawowy interfejs służący do aktualizacji stanu maszyny w trakcie budowy harmonogramu.
Ich zadaniem jest modyfikacja kluczowych atrybutów obiektu, takich jak bieżący stan pracy `state` oraz najwcześniejszy możliwy moment rozpoczęcia kolejnej operacji `available_time`.

Atrybut `available_time` pełni funkcję znacznika czasowego, który gwarantuje poprawną chronologię wykonywanych operacji oraz zapobiega nakładaniu się zadań na tej samej maszynie. Z kolei zmienna `processed_jobs` jest inkrementowana po zakończeniu każdej operacji i wykorzystywana w etapach objętych efektem uczenia się do dynamicznej modyfikacji czasu przetwarzania.

Dzięki tym metodom warstwa modelująca pracę maszyn zachowuje spójność czasową i umożliwia poprawne wyznaczanie harmonogramu w środowisku Hybrid Flow Shop z przezbrojeniami zależnymi od kolejności oraz efektem uczenia się.

```
def compute_processing_time(self, base_time: float, learning_coeff:
    float = 0.0):
    if not self.is_learning or learning_coeff == 0.0:
        return base_time
    # r = processed_jobs + 1 (następne zadanie będzie r-tym
    wykonaniem)
    r = self.processed_jobs + 1
    # t_r = t_1 * r^(-\alpha)
    effective_time = base_time * (r ** (-learning_coeff))
    return effective_time
```

Listing 5.8: Obliczanie efektywnego czasu przetwarzania (efekt uczenia się).

Kluczowym mechanizmem klasy `Machine` jest implementacja efektu uczenia się, zaprezentowanego w listingu 5.8, ponieważ tylko obiekt maszyny śledzi liczbę wcześniej wykonanych operacji na danym etapie produkcyjnym. Efektywny czas przetwarzania r-tej operacji na maszynie jest modyfikowany zgodnie z formułą:

$$P'_r = P_{base} \cdot r^{-\alpha} \tag{5.1}$$

gdzie:

- P'_r oznacza efektywny czas przetwarzania (z uwzględnieniem uczenia)
- P_{base} to bazowy czas operacji (pobrany z macierzy `processing_times`) i przekazany przez klasę nadrzędną
- r to pozycja operacji w sekwencji wykonanej na tej maszynie ($r =$ `processed_jobs` $+ 1$)
- α współczynnik uczenia się, którego wartość mieści się w zakresie $\alpha \in (0, 1]$ i określa tempo redukcji czasu.

Klasa `Job`

Klasa `Job` stanowi podstawowy kontener danych w systemie, reprezentujący pojedyncze zadanie w środowisku Hybrid Flow Shop (HFS) z przezbrojeniami zależnymi od kolejności (SDST) oraz efektem uczenia się.

```
def __init__(self, job_id, stages, setup_times, name=None):
    self.id = job_id
    self.stages = stages
    self.setup_times = setup_times
    self.machine_assignment = [-1 for _ in range(len(stages))]
    self.completion_times = [0.0 for _ in range(len(stages))]
    self.start_processing_times = [0.0 for _ in range(len(stages))]
    self.start_setup_times = [0.0 for _ in range(len(stages))]
    self.end_setup_times = [0.0 for _ in range(len(stages))]
    self.total_time = 0.0
    self.name = name or f"Job_{job_id}"
```

Listing 5.9: Konstruktor obiektu `Job`.

Głównym zadaniem obiektu `Job` jest przechowywanie wszystkich parametrów wejściowych, co widoczne jest w atrybutach `stages` (bazowe czasy przetwarzania) i `setup_times` (macierze przezbrojeń). Jednocześnie, w celu gromadzenia wyników symulacji, konstruktor przedctawiony na Listingu 5.9 inicjalizuje atrybuty wynikowe, takie jak przydział maszyn oraz wszystkie znaczniki czasowe (`completion_times`, `start_processing_times`). Atrybuty te są dynamicznie wypełniane w trakcie procesu symulacji tworzenia harmonogramu, co pozwala na łatwe śledzenie i wizualizację ostatecznego czasu zakończenia zadania $\mathbf{C_j}$ (`total_time`).

```
def get_processing_time(self, stage: int, machine: int) -> float:
    base_time = self.stages[stage][machine]
    if isinstance(base_time, list):
        base_time = base_time[0]
    return float(base_time)
```

Listing 5.10: Metoda zwracająca czas wykonania zadania na danym etapie oraz maszynie.

Metoda `get_processing_time` (Listing 5.11) odpowiada za bezpieczny odczyt bazowego czasu przetwarzania p_{jak} z macierzy `self.stages`. Logika zawarta w listingu umożliwia pobranie wartości niezależnie od tego, czy dane wejściowe zostały dostarczone jako pojedyncza liczba, czy w formacie listy, co zwiększa odporność systemu na zróżnicowane formaty instancji problemu.

```
def get_setup_time(self, prev_job: int, stage: int,
                                    machine: int) -> float:
    if prev_job is None or prev_job == -1:
        setup_val = self.setup_times[self.id][self.id][stage][
    machine]
    else:
        setup_val = self.setup_times[prev_job][self.id][stage][
    machine]
    if isinstance(setup_val, list):
        setup_val = setup_val[0]
    return float(setup_val)
```

Listing 5.11: Metoda zwracająca czas przezborjenia z danego etapu, maszyny oraz poprzedniego zadania.

Metoda `get_setup_time` (Listing 5.11) realizuje istotny mechanizm przezbrojeń zależnych od kolejności. Przyjmuje ona identyfikator poprzedniego zadania (`prev_job`) i na tej podstawie zwraca wymagany czas przezbrojenia. Warunek `if prev_job is None or prev_job == -1` jest niezbędny do obsługi wstępnego przezbrojenia, czyli sytuacji, gdy maszyna jest bezczynna i musi się przygotować do wykonania pierwszego zadania.

```
def set_stage_times(self, stage, machine, setup_start, setup_end,
    proc_start, proc_end):
    if proc_start < setup_end:
        raise ValueError(
            f"[Job {self.id}] Czas rozpoczęcia przetwarzania ({
    proc_start}) "
            f"nie może być wcześniejszy niż koniec przezbrojenia ({
    setup_end}) na etapie {stage}."
        )
    if proc_end < proc_start:
        raise ValueError(
            f"[Job {self.id}] Czas zakończenia etapu ({proc_end}) "
            f"nie może być wcześniejszy niż czas rozpoczęcia
    przetwarzania ({proc_start})."
        )
    self.start_setup_times[stage] = setup_start
    self.end_setup_times[stage] = setup_end
    self.start_processing_times[stage] = proc_start
    self.completion_times[stage] = proc_end
    self.machine_assignment[stage] = machine
    self.total_time = max(self.completion_times)
```

Listing 5.12: Metoda ustawiająca wszystkie czasy związane z przetwarzaniem i przezbrojeniem na danym etapie.

Metoda `set_stage_times` (Listing 5.12) służy do trwałego zapisania wszystkich znaczników czasowych i przydziału maszyny po obliczeniu operacji przez moduł planujący. Jest to jedyna metoda, która aktualizuje stan wynikowy obiektu `Job`.

W pierwszej kolejności metoda wymusza walidację poprawności chronologicznej za pomocą dwóch warunków:

1. zapewnia, że czas rozpoczęcia przetwarzania (`proc_start`) nie jest wcześniejszy niż czas zakończenia przezbrojenia (`setup_end`)
2. gwarantuje, że czas zakończenia etapu (`proc_end`) następuje po czasie jego rozpoczęcia

Następnie, po pozytywnej walidacji, następuje zapis wszystkich czasów i przydziału maszyny (`machine_assignment`). Na koniec atrybut `total_time` jest aktualizowany do maksymalnej wartości czasu zakończenia etapów, co reprezentuje czas zakończenia zadania C_j.

Główne Funkcjonalności Klasy `Schedule`

Posiadając pełen kontekst dotyczący działania poszczególnych klas `Job`, `Machine` oraz dostępnych w nich metodach można przejść do kluczowej funkcji klasy odpowiedzialnej za cały harmonogram `Schedule` — `build_from_sequence`. Metoda ta jest modułem, który przetwarza zadaną permutację zadań π na mierzalny harmonogram, dążący do minimalizacji czasu wykonania C_{max}.

1. **Kryterium Wyboru Maszyny**: W procesie przydzielania zadań do etapu z równoległymi maszynami stosowana jest reguła priorytetu oparta na kalkulacji całkowitego czasu zakończenia. Jej celem jest wybranie maszyny minimalizującej całkowity czas zakończenia operacji, obliczanego jako suma czasu dostępności maszyny, czasu przezbrojenia oraz czasu przetwarzania.
   (a) Główna zasada: System wybiera maszynę minimalizującą ten wynik.
   (b) Rozstrzyganie Remisów: Jeśli maszyny mają identyczny czas dostępności, reguła wybiera tę, która posiada najkrótszą sumę czasu przezbrojenia i przetwarzania.
2. **Ograniczenia Technologiczne**: Metoda `build_from_sequence` egzekwuje warunek konieczny dlahybrydowego przepływowego problemu szeregowania zadań, wyznaczając czas rozpoczęcia przetwarzania (`proc_start`):

$$proc_start = \max(setup_end, prev_stage_end) \tag{5.2}$$

   Powyższe równanie gwarantuje, że przetwarzanie rozpocznie się dopiero po zakończeniu przezbrojenia maszyny (`setup_end`) oraz po zakończeniu operacji na poprzedzającym etapie (`prev_stage_end`), co zapewnia poprawny przepływ zadań.

```python
def build_from_sequence(self, sequence):
    self._reset_runtime_state()
    jobs, machines = self.jobs, self.machines
    mps, S = self.machines_per_stage, self.num_stages
    for j in sequence:
        job = jobs[j]
        prev_stage_end = 0.0
        for s in range(S):
            if mps[s] >= 2:
                m = min(range(mps[s]), key=lambda k: self._calculate_total_time(job, s, k, machines))
            else:
                m = 0
            machine = machines[s][m]
            prev_job = machine.last_job
            setup_dur = job.get_setup_time(prev_job, s, m)
            setup_start = machine.available_time
            setup_end = machine.start_setup(setup_start, setup_dur)
            base_time = job.get_processing_time(s, m)
            proc_time = machines[s][m].compute_processing_time(base_time, self.learning_coeff)
            proc_start = max(setup_end, prev_stage_end)
            proc_end = machine.start_processing(job.id, proc_start, proc_time)
            job.set_stage_times(s, m, setup_start, setup_end, proc_start, proc_end)
            self.add_operation(
                "setup",
                from_job=(prev_job if prev_job is not None else job.id),
                to_job=job.id,
                stage=s,
                machine=m,
                start=setup_start, end=setup_end
            )
            self.add_operation(
                "operation",
                job=job.id,
                stage=s,
                machine=m,
                start=proc_start, end=proc_end
            )
            prev_stage_end = proc_end
    self.Cmax = self.compute_Cmax()
    return self.Cmax
```

Listing 5.13: Kluczowa metoda budująca gotowy harmonogaram produkcji z podanej permutacji.

```
def _calculate_total_time(self, job, stage, machine_idx, machines):
    machine = machines[stage][machine_idx]
    prev_job = machine.last_job
    setup_dur = job.get_setup_time(prev_job, stage, machine_idx)
    base_time = job.get_processing_time(stage, machine_idx)
    proc_time = machine.compute_processing_time(base_time, self.
   learning_coeff)
    return machine.available_time + setup_dur + proc_time
```

Listing 5.14: Metoda odpowiadająca za obliczenie czasu przetwarzania zadania na danej maszynie

Metoda `build_from_sequence`, przedstawiona w Listingu 5.13, stanowi jądro modułu planującego i realizuje proces symulacji przebiegu produkcji. Logika ta opiera się na dwóch zagnieżdżonych pętlach, które iteracyjnie budują harmonogram zadanie po zadaniu (π) i etap po etapie (S). Poniżej omówiono szczegóły implementacji kluczowych kroków algorytmu.

Pierwszym krokiem jest zresetowanie obecnej permutacji oraz przypisanie atrybutów klasy `Schedule` w celu łatwiejszego operowania wewnątrz metody.

Przechodząc do wnętrza pętli, najpierw wybierana jest maszyna 5.1.3, na której wykonuje się dane zadanie w danym etapie. Zastosowany tam warunek sprawdza, czy jest wiele maszyn. W sytuacji, gdy występuje tylko jedna maszyna, jest ona wybierana. W etapach z wieloma maszynami wybierana jest maszyna, która szybciej zakończy przetwarzanie zadania przy użyciu wyrażenia *lambda* i funkcji `_calculate_total_time`, przedstawionej na listingu 5.14, która oblicza całkowity czas zakończenia zadania na danej maszynie. Funkcja ta wybiera maszyne, która zakończy na danym etapie najszybciej; ma to kluczowy wpływ w przypadkach, gdy czas dostępności jest identyczny.

Następnie obliczany jest czas rozpoczęcia oraz zakończenia przezbrojenia i realizacji zadania. Stosując mechanikę klasy `Machine`, ustalany jest czas zakończenia przezbrojenia. Następnie obliczany jest efektywny czas przetwarzania zadania poprzez wywołanie `compute_processing_time` z obiektu `Machine`. Pozwala to bez problemu zastosować wybrane ograniczenia problemu HFS, takie jak efekt uczenia się (realizowany w `Machine`) oraz zależności czasów przezbrojeń (SDST) (realizowany w `Job`).

Najważniejszym elementem jest wyznaczenie czasu rozpoczęcia przetwarzania, który jest wynikiem zastosowania funkcji max: `proc_start = max(setup_end, prev_stage_end)`. W ten sposób algorytm gwarantuje, że zadanie rozpocznie operację dopiero po zakończeniu przezbrojenia maszyny oraz po zakończeniu operacji na poprzedzającym etapie.

Po ustaleniu czasów przetwarzania zadania na danym etapie dodawane są informacje o zadaniu do tablicy `schedule`, która odpowiada za tworzenie gotowej instancji harmonogramu w celu późniejszej analizy lub zapisu. Maszyna jest jednocześnie aktualizowana (`start_processing`) w celu prawidłowego ustalenia jej nowego czasu dostępności (`available_time`) oraz licznika wykonanych zadań.

Na podobnej zasadzie działa funkcja `build_natural_schedule`, która buduje harmonogram na permutacji naturalnej podanej przez użytkownika. Służyła ona w celu weryfikacji, czy zaimplementowany algorytm działa poprawnie i zmniejsza C_{max}.

Oprócz metod zarządzających procesem szeregowania, klasa `Schedule` jest wyposażona w funkcje do obsługi danych, obliczeń i eksportu, niezbędne dla pełnego cyklu życia harmonogramu. Poniżej przedstawiono ich zestawienie:

- `compute_Cmax` – Metoda oblicza ostateczny czas zakończenia całego harmonogramu (C_{max}). W tym celu funkcja odszukuje maksymalny czas zakończenia ze wszystkich etapów, we wszystkich zadaniach.
- `add_operation` – łuży do dodawania informacji o każdym zdarzeniu (zarówno przezbrojenie, jak i przetwarzanie) do wewnętrznej tablicy `schedule`. Dane te stanowią później bazę do generowania wizualizacji (Diagram Gantta) oraz eksportu.
- `to_json` – Metoda formatuje pełny stan harmonogramu (czasy, przypisania maszyn, C_{max}) do postaci tekstowej w formacie `*.JSON`, przygotowując dane do zapisu, zgodnego z założoną strukturą wyjściową opisaną w rozdziale 4.3.
- `save_to_file` – Odpowiada za fizyczny zapis sformatowanych danych (z użyciem `to_json`) do pliku na dysku, umożliwiając trwałe przechowanie rozwiązania.

5.1.4. Implementacja Algorytmów

Posiadając informacje o tym, jak działa klasa `Schedule` oraz w jaki sposób budowany jest harmonogram z zadanej permutacji, można przejść do omówienia zaimplementowanych algorytmów optymalizacyjnych.

Algorytmy te stanowią nadrzędny mechanizm, którego zadaniem jest generowanie najlepszej możliwej permutacji zadań (π^*), minimalizującej funkcję celu harmonogramu, czyli całkowity czas wykonania ($\mathbf{C_{max}}$).

```
class BaseAlgorithm(ABC):
    def __init__(self, schedule, name="BaseAlgorithm"):
        self.name, self.schedule = name, schedule
        self.best_sequence = None
        self.best_cmax = None
        self.result_schedule = None
        self.runtime = 0.0
    @abstractmethod
    def run(self):
        pass
    def get_results(self):
        return {
            "C_max": self.best_cmax,
            "time_in_ms": round(self.runtime * 1000, 3),
            "schedule": self.result_schedule
        }
```

Listing 5.15: Klasa abstrakcyjna `BaseAlgorithm`

Klasa `BaseAlgorithm` (Listing 5.15) jest zaimplementowana jako abstrakcyjna klasa bazowa (ABC), dziedzicząca po `abc.ABC` i stanowiąca fundament dla wszystkich algorytmów optymalizacyjnych. Jej główną rolą jest zdefiniowanie szablonu i wspólnego stanu

dla metod optymalizacyjnych. Za pomocą dekoratora `@abstractmethod` wymusza na klasach pochodnych obowiązkową implementację kluczowej funkcji `run()`, w której musi być zawarta cała logika znajdowania lub tworzenia lepszego rozwiązania. Ponadto, `BaseAlgorithm` przechowuje i inicjuje kluczowe atrybuty wspólne dla wszystkich algorytmów, takie jak: obiekt harmonogramu (`schedule`), najlepszą sekwencję (`best_sequence`), najlepszy wynik (C_{max}), czas wykonania (`runtime`) oraz finalny harmonogram (`result_schedule`), a także dostarcza wspólną metodę `get_results()` do ustandaryzowanego raportowania wyników.

Algorytm dokładny – Branch and Bound

Algorytm *Branch and Bound* (B&B) należy do grupy metod dokładnych, których głównym celem jest gwarancja znalezienia globalnego optimum (π^*) problemu. Ze względu na to, że przeszukuje on całą przestrzeń rozwiązań, jest metodą najbardziej obciążającą zasoby komputera – złożoność obliczeniowa B&B w najgorszym przypadku jest eksponencjalna.
Z tego powodu odpowiednia implementacja jest kluczowa i musi koncentrować się na maksymalnym skróceniu czasu przeszukiwania.

```
class BranchAndBound(BaseAlgorithm):
    def __init__(self, schedule):
        super().__init__(schedule, name="Branch and Bound")
        self.ub_start = math.inf
}
```

Listing 5.16: Inicjalizacja klasy odpowiedzialnej za *B&B*

Aby uruchomić algorytm, należy stworzyć jego instancje. Podczas jej tworzenia (listing 5.16) jest inicjalizowany *Upper Bound*, który będzie modyfikowany.

```
def compute_lower_bound(self, partial_seq, remaining_jobs):
    if not partial_seq:
        return 0.0
    temp = deepcopy(self.schedule)
    current_cmax = temp.build_from_sequence(partial_seq)
    return current_cmax
```

Listing 5.17: Medoda odpowiadająca za obliczenie *dolnego ograniczenia*

Kluczowym elementem jest funkcja zaprezentowana na listingu 5.17. Odpowiada ona za obliczenie *Lower Bound*, zgodnie z podejściem zdefiniowanym w sekcji 4.2.1, które dla problemu HFS-SDST przyjmuje postać:

$$\mathrm{LB}(\pi') = C_{\max}(\pi') \tag{5.3}$$

gdzie $C_{\max}(\pi')$ oznacza rzeczywisty maksymalny czas zakończenia dla harmonogramu zbudowanego na podstawie częściowej permutacji π', z pełnym odzwierciedleniem ograniczeń logicznych oraz zależności sekwencyjnych w czasach przezbrojeń.

Realizację tego mechanizmu przedstawia metoda `compute_lower_bound`. Funkcja ta:

1. *Buduje rzeczywisty harmonogram*: Wykorzystuje `build_from_sequence(partial_seq)` do utworzenia pełnego harmonogramu dla zadań z częściowej sekwencji π'.
   Funkcja ta uwzględnia wszystkie ograniczenia problemu, w tym:
   (a) kolejność procesów – zadanie musi zakończyć etap s zanim rozpocznie etap $s+1$,
   (b) dostępność maszyn – uwzględnia czasy zakończenia poprzednich operacji,
   (c) czasy przezbrojeń – zależne od kolejności zadań $s_{i,j}$,
   (d) efekt uczenia – redukcja czasu przetwarzania na etapach uczących się.
2. *Zwraca poprawne dolne ograniczenie* (Linia 6): Wartość $C_{\max}(\pi')$ stanowi dopuszczalne dolne ograniczenie (*admissible lower bound*) – gwarantuje, że żadne rozszerzenie częściowej sekwencji π' nie może dać rozwiązania o wartości $C_{\max}$ mniejszej niż $\mathrm{LB}(\pi')$.

Takie podejście zapewnia poprawność algorytmu Branch & Bound i gwarantuje znalezienie rozwiązania optymalnego dla rozważanego problemu, co jest kluczowe dla pełnej weryfikacji.

Metoda `run()` jest nadrzędną funkcją, która aktywuje proces przeszukiwania przestrzeni rozwiązań. Stanowi ona integrację wszystkich omówionych mechanizmów algorytmu Branch and Bound. Wykorzystuje ona strategię przeszukiwania w głąb (DFS, *depth-first search*), zarządzając węzłami za pomocą stosu (`stack`), a jej głównym celem jest iteracyjne rozwijanie drzewa i stosowanie reguły ograniczania (Bounding) opartej na funkcji LB, aby zminimalizować czas działania.

```
def run(self):
    start_time = time.time()
    n = len(self.schedule.jobs)
    temp = deepcopy(self.schedule)
    ub = temp.build_natural_schedule()
    best_cmax = ub
    best_seq = list(range(n))
    stack = [([], list(range(n)))]
    while stack:
        partial, remaining = stack.pop()
        if not remaining:
            temp = deepcopy(self.schedule)
            cmax = temp.build_from_sequence(partial)
            if cmax < best_cmax:
                best_cmax = cmax
                best_seq = partial
            continue
        lb = self.compute_lower_bound(partial)
        if lb >= best_cmax:
            continue  # odcinamy gałąź
        for j in remaining:
            new_partial = partial + [j]
```

```
23                new_remaining = [r for r in remaining if r != j]
24                stack.append((new_partial, new_remaining))
25        self.best_sequence = best_seq
26        self.best_cmax = best_cmax
27        end_time = time.time()
28        temp = deepcopy(self.schedule)
29        temp.build_from_sequence(best_seq)
30        self.result_schedule = temp.to_json()
31        self.runtime = end_time - start_time
32        return self.get_results()
```

Listing 5.18: Medoda do uruchomienia algorytmu *Branch and Bound*

Kod na listingu 5.18 przedstawia sposób, w jaki konstruowane jest optymalne rozwiązanie. W pierwszej kolejności wyliczane jest Górne Ograniczenie (UB) z permutacji naturalnej (linia 4), co stanowi szybką, wstępną wartość referencyjną dla późniejszego ograniczania. Następnie tworzony jest stos (`stack`, linia 8), który umożliwia przeszukiwanie przestrzeni rozwiązań w głąb (DFS).

Po przygotowaniu informacji można przejść do budowania optymalnej permutacji. Odpowiada za to główna pętla `while stack:` (linia 9), która iteracyjnie przetwarza węzły drzewa. Najpierw pobierana jest informacja o pierwszym węźle (linia 10); jest to początek przeszukiwania.

Następnie sprawdzane jest, czy powstała już cała permutacja, jeżeli tak, to budowany jest pełen harmonogram o zadanej permutacji oraz wyliczane jest całkowity czas zakończania realkizacji zadań C_{max}. Jeżeli dana permutacja jest lepsza od górnego ograniczenia, to zapisywane są informacje o tej permutacji oraz o najlepszym dotychczasowym czasie zakończenia (linie 11-17). Użycie `continue` powoduje natychmiastowe przerwanie bieżącej iteracji pętli `while` i pominięcie dalszej logiki.

W przypadku gdy nie ma jeszcze pełnego rozwiązania, algorytm oblicza dolne ograniczenie z wykorzystaniem `compute_lower_bound`. Porównuje je z górnym ograniczeniem. Sprawdzany jest wówczas warunek odcięcia gałęzi. (linia 18 - 20).

Po pomyślnym przejściu testu ograniczania algorytm przechodzi do Rozgałęziania (Branching), za które odpowiada pętla `for j in remaining` (Linie 21–24). Pętla ta iteruje po wszystkich zadaniach (`j`), które pozostały do przypisania, tworząc i generując wszystkie możliwe bezpośrednie rozszerzenia bieżącej częściowej permutacji. W każdej iteracji tworzony jest nowy węzeł (`new_partial`, `new_remaining`), który jest dodawany do stosu (`stack.append`), kontynuując w ten sposób przeszukiwanie w głąb na kolejnym poziomie drzewa.

Po zakończeniu pętli `while` wszelkie informacje o optymalnym rozwiązaniu są ostatecznie rejestrowane. Najlepsza znaleziona sekwencja (`best_seq`) oraz ostateczny C_{max} są zapisywane w atrybutach klasy. Następnie algorytm oblicza całkowity czas wykonania (`runtime`) i dekoduje najlepszą sekwencję, aby stworzyć gotowy harmonogram. Ostatecznie funkcja `get_results()` zwraca ustandaryzowany zbiór danych w formacie słownika, który jest przygotowany do zapisu w formacie `*.JSON`, zawierający wszystkie niezbędne informacje o rozwiązaniu.

Algorytm heurystyczny – Min Setup Time First

Algorytm *Min Setup Time First* jest heurystyką konstrukcyjną. Jego celem jest szybkie zbudowanie wysokiej jakości permutacji zadań π, która minimalizuje koszty przezbrojeń zależnych od kolejności, zmniejszając przy tym całkowity czas realizacji zadań C_{max}.

```
class MinSTF(BaseAlgorithm):
    def __init__(self, schedule):
        super().__init__(schedule, name="MinSTF")
```

Listing 5.19: Inicjalizacja klasy odpowiedzialnej za MinSTF

Listing 5.19 przedstawia inicjalizację algorytmu minimalizującego średni czas przezbrojeń. Strategia ta dąży do skrócenia całkowitego czasu realizacji wszystkich zadań.

```
1  def _calculate_setup_time(self, job_id, prev_job_id=None):
2      if prev_job_id is None:
3          prev_job_id = job_id
4      job = self.schedule.jobs[job_id]
5      setup_times = [
6          job.get_setup_time(prev_job_id, stage, machine)
7          for stage in range(self.schedule.num_stages)
8          for machine in range(self.schedule.machines_per_stage[stage
       ])
9       ]
10     return sum(setup_times) / len(setup_times) if setup_times else
       0.0
```

Listing 5.20: Medoda do obliczenia średniego czasu przezbrojeń

Metoda `_calculate_setup_time` (listing 5.20) odpowiada za obliczanie średniego czasu przezbrojenia pomiędzy dwoma zadaniami. Uśrednienie jest kluczowe, ponieważ problem jest wielomaszynowy, a heurystyka musi ocenić globalny koszt sekwencji na wszystkich etapach. Implementacja jest uniwersalna:

- Linie 2–3: Warunek `if prev_job_id is None` obsługuje wybór zadania startowego, ustawiając przezbrojenie jako self-setup (z zadania J_j na J_j).
- Linie 5–9 odpowiadają za zbieranie czasów przezbrojeń dla każdego etapu oraz maszyny.

Wyodrębnienie tej funkcji pozwala na oddzielenie logiki obliczeniowej od logiki decyzyjnej algorytmu. Metoda ta dostarcza jasny, liczbowy wskaźnik do metody `run()`, która używa go do podjęcia zachłannej decyzji o wyborze kolejnego zadania.

```
def run(self):
    start_time, n_jobs = time.time(), len(self.schedule.jobs)
    remaining_jobs, sequence = set(range(n_jobs)), []
    # Krok 1: Wybierz pierwsze zadanie (najmniejszy self-setup)
    min_self_setup, first_job = float('inf'), -1
    for candidate in remaining_jobs:
        avg_setup = self._calculate_setup_time(candidate)
        if avg_setup < min_self_setup:
            min_self_setup, first_job = avg_setup, candidate
    sequence.append(first_job)
    remaining_jobs.remove(first_job)
    # Krok 2: Iteracyjnie wybieraj zadania o minimalnym setupie
    while remaining_jobs:
        last_job = sequence[-1]
        min_setup = float('inf')
        best_job = None
        for candidate in remaining_jobs:
            avg_setup = self._calculate_setup_time(candidate,
   last_job)
            if avg_setup < min_setup:
                min_setup, best_job = avg_setup, candidate
        sequence.append(best_job)
        remaining_jobs.remove(best_job)
    self.runtime = time.time() - start_time
    cmax = self.schedule.build_from_sequence(sequence)
    self.best_sequence = sequence
    self.best_cmax = cmax
    self.result_schedule = self.schedule.to_json()
    return self.get_results()
```

Listing 5.21: Medoda do uruchomienia algorytmu *Minimum Setup Time First*

Kod na listingu 5.21 przedstawia całość logiki budowania permutacji w sposób heurystyczny. W pierwszej kolejności uruchamiamy mierzenie czasu, pobieramy liczbę zadań, a następnie inicjujemy zbiór zadań do uporządkowania oraz pustą listę permutacji.

Następnie wybierane jest pierwsze zadanie (Linie 17–36). Wybór ten polega na obliczeniu średniego czasu przezbrojenia zadania z samego na siebie (self-setup) za pomocą funkcji `_calculate_setup_time(candidate)`.

Pętla `for` przeszukuje wszystkie zadania i wybiera to, które minimalizuje ten koszt. W ten sposób pierwsze zadanie w gotowym harmonogramie ma najkrótszy średni, wewnętrzny czas przezbrojenia. Wybrane zadanie jest dodawane do gotowej permutacji (`sequence`), a następnie usuwane ze zbioru zadań pozostałych do przypisania (`remaining_jobs`).

Pętla `while remaining_jobs:` (Linia 39) jest kluczowym elementem algorytmu, który odpowiada za iteracyjne rozszerzanie permutacji do momentu, aż wszystkie zadania zostaną przypisane. Zasada działania pozostaje ta sama, jak podczas wyboru pierwszego zadania w permutacji z tą różnicą, że posiadamy informacje o poprzednim zadaniu. Po zakończeniu tej pętli permutacja jest zapisywana, a całkowity czas realizacji harmonogramu jest obliczany.

Algotym Metaheurystyczny – Tabu Search

Kompromisem pomiędzy czasem realizacji a jakością wyniku są algorytmy metaheurystyczne. Algorytmy te, w przeciwieństwie do algorytmów dokładnych (jak B&B z sekcji 5.1.4), mają na celu znalezienie wysokiej jakości rozwiązania w rozsądnym czasie, intensywnie przeszukując przestrzeń rozwiązań.

```
class TabuSearch(BaseAlgorithm):
    def __init__(self, schedule, max_iter=100, tabu_tenure=10,
   no_improve_limit=10):
        super().__init__(schedule, name="Tabu-Search")
        self.max_iter = max_iter
        self.tabu_tenure = tabu_tenure
        self.no_improve_limit = no_improve_limit
        self.tabu_list = {}
        self.iterations_performed = 0
        self.iterations_without_improvement = 0
```

Listing 5.22: Inicjalizacja klasy odpowiedzialnej za `Tabu Search`

Listing 5.22 przedstawia inicjalizację klasy `TabuSearch`. Ponieważ Tabu Search jest metaheurystyką, jej skuteczność zależy od odpowiedniego doboru parametrów kontrolnych, definiowanych w konstruktorze. Konstruktor ten umożliwia intuicyjną zmianę tych parametrów.

```
def _generate_initial_solution(self):
    minstf = MinSTF(self.schedule)
    minstf.run()
    return minstf.best_sequence
```

Listing 5.23: Medoda do generowania *początkowego rozwiązania*

Listing 5.23 przedstawia implementację metody `_generate_initial_solution()`. Metoda ta rozwiązuje problem za pomocą heurystyki konstrukcyjnej Min Setup Time First (MSTF), która została szczegółowo opisana powyżej 5.1.4.

```
def _generate_neighborhood(self, sequence):
   n = len(sequence)
   neighbors = []
   for i in range(n):
       for j in range(i + 1, n):
           neighbor = sequence.copy()
           neighbor[i], neighbor[j] = neighbor[j], neighbor[i]
           neighbors.append((neighbor, (i, j)))
   return neighbors
```

Listing 5.24: Medoda do *generacji sąsiedztwa*

Metoda `_generate_neighborhood` generuje przestrzeń sąsiedztwa dla algorytmu Tabu Search. Zastosowano ruch typu `swap`, który polega na systematycznej zamianie pozycji dwóch dowolnych zadań w bieżącej sekwencji.Zagnieżdżone pętle `for` odpowiadają za iteracyjne przeszukiwanie wszystkich możliwych par indeksów (i, j) (gdzie $i < j$), generując nowe sekwencje (sąsiadów).Wynikiem operacji jest zbiór par (`neighbor`, (i, j)), gdzie informacja o

ruchu (i, j) jest krytyczna do zarządzania listą Tabu.Złożoność generowania otoczenia wynosi $O(n^2)$, gdyż jego liczność to $\frac{n(n-1)}{2}$. Ta kwadratowa złożoność jest niezbędna, aby zapewnić dokładne przeszukanie całego otoczenia w każdej iteracji i wybranie najlepszego sąsiada spełniającego kryteria Tabu Search.

```
def _select_best_neighbor(self, neighbors, current_sequence, global_best_cmax, current_iteration):
    best_neighbor = None
    best_cmax = float('inf')
    best_move = None
    for neighbor_seq, move in neighbors:
        if self._is_tabu(move, current_iteration):
            neighbor_cmax = self.schedule.build_from_sequence(neighbor_seq)
            if neighbor_cmax < global_best_cmax:
                if neighbor_cmax < best_cmax:
                    best_neighbor = neighbor_seq
                    best_cmax = neighbor_cmax
                    best_move = move
            continue
        neighbor_cmax = self.schedule.build_from_sequence(neighbor_seq)
        if neighbor_cmax < best_cmax:
            best_neighbor = neighbor_seq
            best_cmax = neighbor_cmax
            best_move = move
    if best_neighbor is None:
        for neighbor_seq, move in neighbors:
            neighbor_cmax = self.schedule.build_from_sequence(neighbor_seq)
            if neighbor_cmax < best_cmax:
                best_neighbor = neighbor_seq
                best_cmax = neighbor_cmax
                best_move = move
    return best_neighbor, best_cmax, best_move
```

Listing 5.25: Medoda do wyznaczenia *najlepszego siąsiada*

Po wygenerowaniu otoczenia metoda `_select_best_neighbor` (listing 5.25) odpowiada za kluczową fazę oceny i wyboru najlepszego kandydata do wykonania ruchu. W ten sposób algorytm Tabu Search przeszukuje przestrzeń rozwiązań, dążąc do minimalizacji C_{max}.

Główna pętla iteruje przez wszystkich potencjalnych sąsiadów, rozpoczynając od sprawdzenia, czy dany ruch `move` jest objęty zakazem Tabu za pomocą funkcji `_is_tabu`. Jeśli ruch jest zakazany, algorytm natychmiast przechodzi do weryfikacji Kryterium Aspiracji, aby sprawdzić możliwość złamania tego zakazu.

Kryterium Aspiracji (Linie 7–13) jest mechanizmem, który pozwala na zaakceptowanie zakazanego ruchu, jeśli prowadzi on do rozwiązania lepszego niż dotychczasowe globalne optimum (`global_best_cmax`). Jeśli ten warunek jest spełniony, ruch jest akceptowany pomimo jego obecności na liście Tabu. Natomiast jeśli ruch nie jest zakazany, jego C_{max} jest porównywany z aktualnie najlepszym znalezionym sąsiadem (`best_cmax`) i, jeśli jest lepszy, zostaje wybrany.

W efekcie metoda zawsze wybiera najlepszego sąsiada, który jest albo niezakazany, albo spełnia Kryterium Aspiracji, zapewniając, że algorytm unika cyklicznego powracania do lokalnych minimów, jednocześnie zachowując możliwość skoku do globalnie lepszego obszaru. Wybrana sekwencja i ruch są zwracane do głównej pętli TS, co pozwala na aktualizację listy Tabu.

Lista Tabu jest kluczowym elementem metaheurystyki Tabu Search, pełniąc rolę pamięci krótkotrwałej. Ma ona za zadanie zapobiegać cyklicznemu powrotowi do wcześniej odwiedzonych (gorszych) rozwiązań, zmuszając algorytm do eksploracji nowych obszarów przestrzeni rozwiązań. Pamięć Tabu jest zaimplementowana w formie słownika, co umożliwia szybkie sprawdzenie (w czasie stałym $O(1)$), czy dana operacja jest zakazana, co znacząco zwiększa wydajność algorytmu.

```
def _is_tabu(self, move, current_iteration):
    i, j = move
    moves_to_check = [(i, j), (j, i)]
    for m in moves_to_check:
        if m in self.tabu_list:
            expiration_iter = self.tabu_list[m]
            if current_iteration < expiration_iter:
                return True
    return False
```

Listing 5.26: Metoda do sprawdzenia czy operacja jest na liscie Tabu

Listing 5.26 przedstawia metodę `_is_tabu`, która weryfikuje, czy dany ruch jest aktualnie zakazany. Ponieważ ruch swap jest symetryczny (zamiana i na j jest logicznie równoważna zamianie j na i), metoda sprawdza obecność obu form ruchu ((i, j) oraz (j, i)) w słowniku `self.tabu_list`. Jeśli ruch jest znaleziony, sprawdzany jest warunek `current_iteration` $<$ `expiration_iter` – jeżeli aktualna iteracja jest mniejsza niż iteracja wygaśnięcia, ruch jest zakazany, a funkcja zwraca `True`.

```
def _add_to_tabu(self, move, current_iteration):
    i, j = move
    expiration_iter = current_iteration + self.tabu_tenure
    self.tabu_list[(i, j)] = expiration_iter
    self.tabu_list[(j, i)] = expiration_iter
    self._clean_tabu_list(current_iteration)
```

Listing 5.27: Metoda dodająca operacje do listy Tabu

Metoda `_add_to_tabu` (listing 5.27) odpowiada za rejestrację nowo wykonanego ruchu w pamięci Tabu. Oblicza ona iterację wygaśnięcia (*expiration_iter*) jako sumę bieżącej iteracji i stałej kadencji Tabu (`self.tabu_tenure`). Obie symetryczne formy ruchu (i, j i j, i) są dodawane do listy z tą samą wartością wygaśnięcia. Ostatnim krokiem jest wywołanie metody czyszczącej, która dba o usunięcie przestarzałych wpisów.

```
def _clean_tabu_list(self, current_iteration):
    expired_moves = [move for move,
                     exp_iter in self.tabu_list.items()
                     if exp_iter <= current_iteration]
    for move in expired_moves:
        del self.tabu_list[move]
```

Listing 5.28: Metoda odpowiadająca za usuwanie elementów z listy Tabu

Za utrzymanie czystości i aktualności listy Tabu odpowiada metoda `_clean_tabu_list` (listing 5.28). Metoda ta identyfikuje i usuwa te wpisy, dla których iteracja wygaśnięcia jest mniejsza lub równa bieżącej iteracji (`current_iteration`). Cykliczne czyszczenie jest kluczowe, ponieważ zapobiega niekontrolowanemu rozrostowi listy Tabu, zachowując przy tym jej dynamiczny charakter i efektywność pamięci.

```
def run(self):
    start_time = time.time()
    current_sequence = self._generate_initial_solution()
    current_cmax = self.schedule.build_from_sequence(
   current_sequence)
    best_sequence, best_cmax = current_sequence.copy(), current_cmax
    for iteration in range(self.max_iter):
        self.iterations_performed = iteration + 1
        neighbors = self._generate_neighborhood(current_sequence)
        current_sequence, current_cmax, move = \
        self._select_best_neighbor(
            neighbors, current_sequence, best_cmax, iteration
        )
        if current_cmax < best_cmax:
            best_sequence = current_sequence.copy()
            best_cmax = current_cmax
            self.iterations_without_improvement = 0
        else:
            self.iterations_without_improvement += 1
        if move:
            self._add_to_tabu(move, iteration)
        if self.iterations_without_improvement >= self.
   no_improve_limit:
            break
    self.best_sequence, self.best_cmax = best_sequence, best_cmax
    self.result_schedule = self.schedule.to_json()
    self.runtime = time.time() - start_time
    return self.get_results()
```

Listing 5.29: Medoda do uruchomienia algorytmu `Taby Search`

Listing 5.29 przedstawia metodę run(), która integruje wszystkie mechanizmy i zawiera główną pętlę sterującą przeszukiwaniem Tabu. Na samym początku uruchamiane jest mierzenie czasu, a następnie generowane jest rozwiązanie początkowe (Linia 3) za pomocą heurystyki (`_generate_initial_solution`). Po wyznaczeniu $\mathbf{C_{max}}$ dla tego rozwiązania jego wartości są przypisywane do aktualnego (`current_sequence`, `current_cmax`) oraz najlepszego dotychczas znalezionego stanu (`best_sequence`, `best_cmax`).

Główna pętla `for iteration in range(self.max_iter):` (Linia 6) steruje procesem przeszukiwania, aż do osiągnięcia maksymalnego limitu iteracji. W każdej iteracji najpierw następuje generowanie całego sąsiedztwa (`neighbors`) za pomocą operacji swap (Linia 8), a następnie z tego otoczenia wyciągany jest najlepszy sąsiad poprzez wywołanie `_select_best_neighbor` (linie 9–11), które uwzględnia zarówno zakazy Tabu, jak i Kryterium Aspiracji.

Po wyborze sekwencja i $\mathbf{C_{max}}$ tego najlepszego sąsiada stają się aktualnym rozwiązaniem (Linie 12–13) dla kolejnej iteracji, co jest kluczowe, ponieważ umożliwia algorytmowi ucieczkę z lokalnego minimum, akceptując gorsze rozwiązania. Następnie sprawdzany jest warunek, czy nowe rozwiązanie jest lepsze od dotychczasowego globalnego optimum.
Jeżeli tak, zmieniane są wartości `best_sequence` i `best_cmax`, a licznik braku poprawy jest resetowany (`self.iterations_without_improvement = 0`). W przeciwnym wypadku licznik ten jest zwiększany.

Ostatnie bloki kodu odpowiadają za zarządzanie listą Tabu i kryteriami zatrzymania. Warunek `if move:` (Linia 21) zapewnia, że jeśli zostanie wykonany ruch, jest on dodawany do listy Tabu (`_add_to_tabu`) z określoną kadencją. Następnie sprawdzany jest warunek zakończenia pętli (Linia 23): pętla zostaje przerwana (`break`), jeśli liczba iteracji bez poprawy globalnego optimum osiągnie limit `self.no_improve_limit`. Po zakończeniu przeszukiwania ostatecznie rejestrowane są najlepsze znalezione wyniki $\mathbf{C_{max}}$ oraz czas działania (`runtime`).

5.1.5. Uruchomienie skryptu Pythona

Miejscem uruchomienia całego skryptu jest plik `main.py`. Odpowiada on za wywołanie odpowiedniego algorytmu, tworzenie wykresów oraz zarządzanie całym cyklem życia aplikacji. Plik ten pełni funkcję głównego kontrolera, integrującego logikę harmonogamowania (`core/`) z algorytmami (`algorithms/`) i narzędziami wizualizacyjnymi (`utils/`).

```
def main():
    parser = argparse.ArgumentParser(
        description="Optymalizacja harmonogramu HFS-SDST",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog="""
Przykłady użycia:
  python main.py # oblicz dla default.json
  python main.py data.json # oblicz dla data.json
  python main.py data.json --plot # oblicz i wygeneruj wykres Gantta
  python main.py --gantt result.json # wygeneruj wykres Gantta z istniejącego harmonogramu
  python main.py --gantt result.json out.png # wygeneruj wykres Gantta do pliku out.png

Uwaga: Algorytm jest określany w pliku JSON (pole "algorithm": "tabu"|"greedy"|"bnb")
        """
    )
    parser.add_argument(
        "--plot",
        action="store_true",
        help="Wygeneruj wykres Gantta"
    )
    parser.add_argument(
        "--gantt",
        metavar="SCHEDULE_FILE",
        help="Generuj tylko wykres Gantta z istniejącego harmonogramu"
    )
    parser.add_argument(
        "input_file", nargs="?",
        default="../data/default.json",
        help="Plik wejsciowy z danymi (domyslnie: ../data/default.json)"
    )
    parser.add_argument(
        "output_file",
        nargs="?",
        help="Plik wyjściowy dla wykresu Gantta (używane z --gantt)"
    )
    args = parser.parse_args()
```

```
    if args.gantt:
        if not os.path.exists(args.gantt):
            print(f"[ERROR] Plik nie istnieje: {args.gantt}")
            sys.exit(1)

        generate_gantt_only(args.gantt, args.input_file if args.input_file != "../data/default.json" else None)
        return
    if not os.path.exists(args.input_file):
        print(f"[ERROR] Plik nie istnieje: {args.input_file}")
        sys.exit(1)
    run_algorithm(
        input_file=args.input_file,
        plot=args.plot
    )
if __name__ == "__main__":
    main()
```

Listing 5.30: Główna funkcja sterująca (`main`) i parsowanie argumentów

Na listingu 5.30 zaprezentowano, w jaki sposób przetwarzane są dane podczas wywołania z różnymi flagami.

W funkcji `main()` główny parser argumentów modułu `argparse` definiuje i przetwarza flagi oraz argumenty pozycyjne, kierując przepływem programu do odpowiedniego trybu działania:

- *Tryb Wizualizacji* (Flaga `--gantt`): Aktywuje `generate_gantt_only`. Tryb ten służy do wczytania istniejącego pliku harmonogramu JSON i wygenerowania na jego podstawie wykresu Gantta, pomijając powtarzanie obliczeń optymalizacyjnych.

- *Tryb Normalny* (Brak flag): Podstawowy tryb pracy, w którym program dąży do znalezienia najlepszego możliwego harmonogramu dla podanego pliku wejściowego.

Dodatkowo flaga `--plot` pozwala użytkownikowi na opcjonalne wyświetlenie wygenerowanego wykresu Gantta na ekranie.

```
def get_results_dir():
    """Zwraca ścieżkę do katalogu results względem lokalizacji skryptu."""
    script_dir = os.path.dirname(os.path.abspath(__file__))
    results_dir = os.path.abspath(os.path.join(script_dir, "..", "results"))
    os.makedirs(results_dir, exist_ok=True)
    return results_dir
```

Listing 5.31: Metoda odpowiedzialna za odpowiednie miejsce zapisu dostępne dla GUI oraz Pythona

Funkcja `get_results_dir` (Listing 5.32) służy do zapisywania generowanych plików wynikowych w miejscu dostępnym zarówno dla modułu obliczeniowego w Pythonie, jak i warstwy klienckiej (GUI/Java).

```python
def generate_gantt_only(input_file, output_file=None):
    print(f"[GANTT] Wczytywanie harmonogramu z: {input_file}")
    try:
        gantt = GanttChart.from_file(input_file)
        gantt.print_info()
        if output_file is None:
            base_name = os.path.splitext(os.path.basename(input_file))[0]
            output_file = os.path.join(get_results_dir(), f"{base_name}_gantt.png")
        print(f"\n[GANTT] Generowanie wykresu Gantta...")
        print(f"[GANTT] Plik wyjściowy: {output_file}")
        saved_path = gantt.save(output_file, show=False, update_source=True)
        if os.path.exists(saved_path):
            file_size = os.path.getsize(saved_path)
            print(f"[SUCCESS] Wykres Gantta zapisany pomyślnie!")
            print(f"[SUCCESS] Lokalizacja: {saved_path}")
            print(f"[SUCCESS] Rozmiar: {file_size / 1024:.1f} KB")
            print(f"[UPDATE] Zaktualizowano plik {input_file} z nową ścieżką do wykresu")
            return saved_path
        else:
            print("[ERROR] Nie udało się utworzyć pliku wykresu")
            sys.exit(1)
    except FileNotFoundError as e:
        print(f"[ERROR] {e}")
        sys.exit(1)
    except ValueError as e:
        print(f"[ERROR] {e}")
        sys.exit(1)
    except json.JSONDecodeError as e:
        print(f"[ERROR] Błąd parsowania JSON: {e}")
        sys.exit(1)
    except Exception as e:
        print(f"[ERROR] Wystąpił błąd: {e}")
        import traceback
        traceback.print_exc()
        sys.exit(1)
```

Listing 5.32: Metoda odpowiadająca za tworzenie diagramu Gantta z gotowego harmonogramu

Funkcja `generate_gantt_only` pełni kluczową rolę w trybie wizualizacji systemu, będąc wywoływana z poziomu głównego kontrolera (`main.py`) przy użyciu flagi `-gantt`. Jej głównym zadaniem jest wygenerowanie wykresu Gantta z istniejącego, ukończonego harmonogramu

zapisanego w formacie JSON, bez konieczności powtarzania czasochłonnych obliczeń optymalizacyjnych. Metoda jest oparta na bloku `try...except`, który implementuje mechanizm odporności na błędy.

```
def run_algorithm(input_file, plot=False):
    print(f"\n[MAIN] Wczytywanie danych z pliku: {input_file}")
    with open(input_file, "r", encoding="utf-8") as f:
        data = json.load(f)
    algorithm = data.get("algorithm", "tabu").lower()
    schedule = Schedule()
    schedule.load_from_json(input_file)
    print(f"[MAIN] Dane wejsciowe:")
    print(f"  - Algorytm: {algorithm}")
    print(f"  - Liczba zadan: {len(schedule.jobs)}")
    print(f"  - Liczba etapow: {schedule.num_stages}")
    print(f"  - Maszyny na etapach: {schedule.machines_per_stage}")
    print(f"  - Wspolczynnik uczenia (alpha): {schedule.learning_coeff}")
    print(f"  - Etapy z uczeniem: {schedule.learning_stages}")
    algo_map = {
        "greedy": ("MinSTF", MinSTF),
        "tabu": ("Tabu Search", TabuSearch),
        "bnb": ("Branch & Bound", BranchAndBound),
        "b&b": ("Branch & Bound", BranchAndBound)
    }
    if algorithm not in algo_map:
        print(f"[ERROR] Nieznany algorytm: {algorithm}")
        print(f"[ERROR] Dostepne algorytmy: greedy, tabu, bnb")
        sys.exit(1)
    algo_name, algo_class = algo_map[algorithm]
    print(f"\n[MAIN] Uruchamianie algorytmu: {algo_name}")
    if algorithm in ["tabu"]:
        algo = algo_class(schedule, max_iter=100, tabu_tenure=10, no_improve_limit=10)
    else:
        algo = algo_class(schedule)
    internal_results = algo.run()
    results = {
        "time_in_ms": internal_results["time_in_ms"],
        "Algorithm": algo_name,
        "C_max": internal_results["C_max"],
        "schedule": internal_results["schedule"]
    }
    print(f"\n[MAIN] Wyniki:")
    print(f"  - Algorytm: {results['Algorithm']}")
    print(f"  - C_max: {results['C_max']:.2f}")
    print(f"  - Czas wykonania: {results['time_in_ms'] / 1000:.3f}s")
```

```
    print(f"  - Sekwencja: {algo.best_sequence}")
    results_dir = get_results_dir()
    gantt_file = os.path.join(results_dir, "result_gantt.png")
    print(f"\n[GANTT] Generowanie diagramu Gantta...")
    print(f"[GANTT] Sciezka do diagramu: {gantt_file}")
    try:
        gantt = GanttChart.from_results(results)
        gantt.save(gantt_file, show=plot, update_source=False)
        print(f"[GANTT] Zapisano diagram do: {gantt_file}")
        results["gant_diagram"] = gantt_file
    except Exception as e:
        print(f"[WARNING] Blad podczas generowania diagramu Gantta: {e}")
        import traceback
        traceback.print_exc()
        results["gant_diagram"] = None
    output_file = os.path.join(results_dir, "result.json")
    with open(output_file, "w", encoding="utf-8") as f:
        json.dump(results, f, indent=4, ensure_ascii=False)
    print(f"\n[OUTPUT] Zapisano wyniki do: {output_file}")
    return results
```

Listing 5.33: Metoda odpowiadająca za wywołanie odpowiedniego algorytmu

Listing 5.33 prezentuje funkcję `run_algorithm`, która stanowi centralny punkt logiczny aplikacji. Odpowiada ona za kompleksową organizację procesu optymalizacji: od wczytania danych, przez dynamiczny wybór algorytmu, aż po końcowy zapis wyników i wizualizację.

5.1.6. Moduł wizualizacji harmonogramu

Z powodu tego, że problem jest skomplikowany, należało zaimplementować cały osobny moduł odpowiedzialny za rysowanie diagramów Gantta.

```
class GanttChart:
    JOB_COLORS = [
        '#FFD700',  # Złoty
        '#87CEEB',  # Błękitny
        '#FF6B6B',  # Czerwony
        '#4ECDC4',  # Turkusowy
        '#95E1D3',  # Miętowy
        '#F38181',  # Różowy
        '#AA96DA',  # Fioletowy
        '#FCBAD3',  # Jasnoróżowy
        '#A8E6CF',  # Jasnozielony
        '#FFD3B6',  # Brzoskwiniowy
        '#FFA07A',  # Łososiowy
        '#98D8C8',  # Seledynowy
    ]
    SETUP_COLOR = '#CCCCCC'
```

```
17     SETUP_EDGE_COLOR = '#666666'
18     SETUP_HATCH = '///'
19     def __init__(self, schedule_data=None):
20         self._raw_data = schedule_data if isinstance(schedule_data,
   dict) else {}
21         self._source_file = None
22         if schedule_data is None:
23             self.schedule = []
24             self.cmax = 0
25             self.algorithm = ''
26         elif isinstance(schedule_data, dict):
27             self.schedule = schedule_data.get('schedule', [])
28             self.cmax = schedule_data.get('C_max', 0)
29             self.algorithm = schedule_data.get('Algorithm') or
   schedule_data.get('algorithm', '')
30         else:
31             self.schedule = schedule_data
32             self.cmax = max(op['end'] for op in self.schedule) if
   self.schedule else 0
33             self.algorithm = ''
34         self.num_stages = 0
35         self.machines_per_stage = {}
36         self._detect_structure()
```

Listing 5.34: Konstruktor klasy `GanttChart` wraz z przypisaniem wartości dla zmiennych statycznych

Listing 5.34 przedstawia wpisane na szywno kolory dla poszczególnych zadań oraz dokładne informacje, jak prezentować przezbrojenie. Dodatkowo zaprezentowany jest konstruktor, który może przyjmować jako argument: gotowy harmonogram czy też całośc pliku wynikowego.

Z powodu tego, że biblioteka `matplotlib` nie jest przystosowana do pracy z tak złożonym problemem, każdy element jest rysowany „ręcznie”. Służą temu metody:

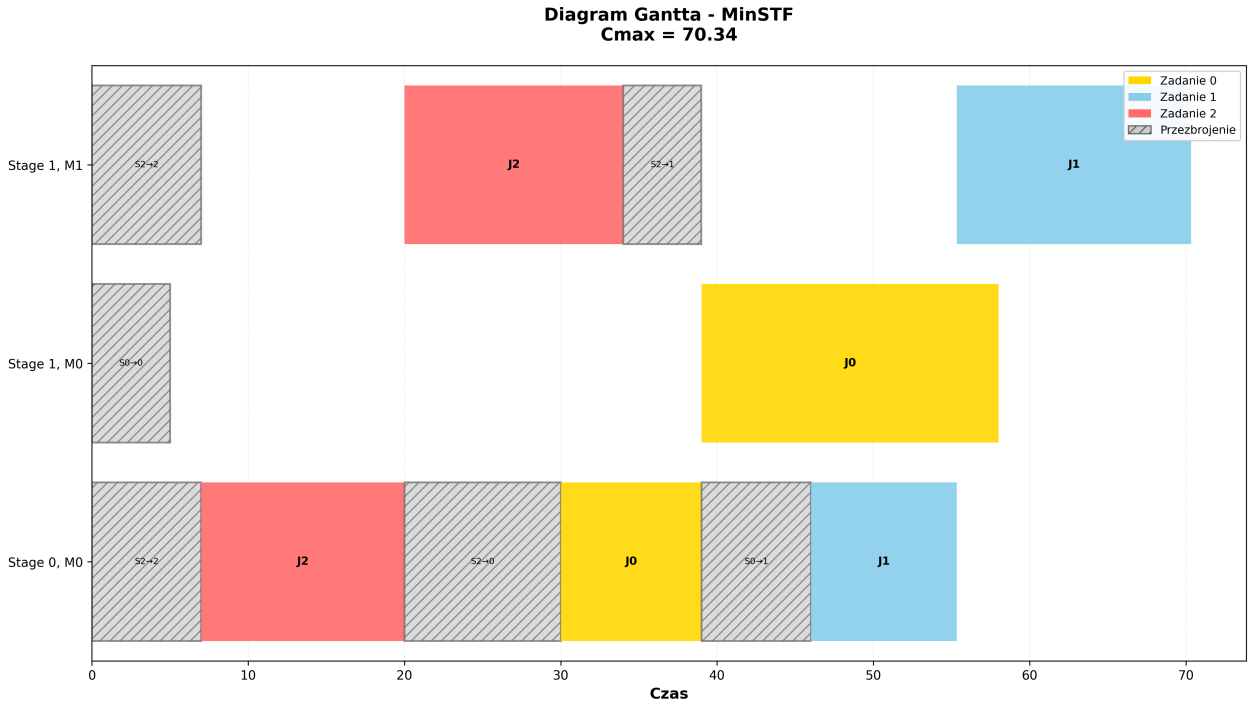
- `generate()` – główna metoda rysująca diagram, iterująca po wszystkich operacjach harmonogramu i tworząca dla każdej z nich obiekt `Rectangle` z biblioteki `matplotlib`
- `_detect_structure()` – automatyczne wykrywanie liczby etapów i maszyn na podstawie danych harmonogramu
- `_create_machine_labels()` – generowanie etykiet osi Y w formacie „Stage X, Machine Y”
- `_get_job_color()` – przypisywanie kolorów do zadań z wykorzystaniem operacji modulo dla cykliczności palety

Wszystkie te metody wspólnie realizują wizualizację, umożliwiając poprawne odwzorowanie złożonego środowiska Hybrid Flow Shop z uwzględnieniem czasów przezbrojenia (SDST) oraz dynamicznego przydziału zadań na poszczególne maszyny równoległe.

```python
def generate(self, output_file=None, figsize=(14, 8), show=True, title=None):
    ... Tworzenie mapy pozycji dla każdej maszyny ...
    for op in self.schedule:
        stage, machine = op['stage'], op['machine']
        start, end = op['start'], op['end']
        duration = end - start
        y_pos = machine_positions[(stage, machine)]
        if op['type'] == 'setup':
            rect = Rectangle(
                (start, y_pos - 0.4), duration, 0.8,
                facecolor=self.SETUP_COLOR,
                edgecolor=self.SETUP_EDGE_COLOR, linewidth=1.5,
                hatch=self.SETUP_HATCH, alpha=0.7
            )
            ax.add_patch(rect)
            from_job  = op.get('from_job', '?')
            to_job = op.get('to_job', '?')
            if duration > 2:
                ax.text(
                    start + duration / 2, y_pos,
                    f'S{from_job}->{to_job}', ha='center',
                    va='center', fontsize=7, color='black',
                    weight='normal'
                )
        elif op['type'] == 'operation':
            job_id, color = op['job'], self._get_job_color(job_id)
            rect = Rectangle(
                (start, y_pos - 0.4), duration, 0.8,
                facecolor=color, edgecolor='none',
                linewidth=0, alpha=0.9
            )
            ax.add_patch(rect)
            if duration > 1.5:
                ax.text(
                    start + duration / 2, y_pos, f'J{job_id}',
                    ha='center', va='center',
                    fontsize=9, color='black', weight='bold'
                )
    ...  Konfiguracja osi oraz dodanie zadań do legendy ...
```

Listing 5.35: Kluczowy fragmęt metody odpowiedzialna za rysowanie diagramu Gantta

Fragment kodu zaprezentowanego w listingu 5.35 przedstawia kluczowy element rysowania diagramu Gantta. Pokazuje on, w jaki sposób tworzone są poszczególne bloki operacji oraz przezbrojeń, które następnie nanoszone są na osie wykresu. Efektem całej procedury jest kompletny i czytelny wykres przedstawiony na rysunku 5.1.



Rysunek 5.1: Przykładowy wykres Gantta dla hybrydowego przepływowego problemu szeregowania zadań

Diagram 5.1 przedstawia dwuetapowy problem dla trzech zadań, gdzie na pierwszym etapie znajduje się jedna maszyna, a na drugim dwie. Efekt uczący jest zaimplementowany na etapie pierwszym.

```
def save(self, output_file, figsize=(14, 8), show=False, title=None, update_source=True):
    output_dir = os.path.dirname(output_file)
    if output_dir: os.makedirs(output_dir, exist_ok=True)
    self.generate(output_file=output_file, figsize=figsize, show=show, title=title)
    abs_output = os.path.abspath(output_file)
    if update_source and self._source_file and os.path.exists(self._source_file):
        try:
            with open(self._source_file, 'r', encoding='utf-8') as f:
                data = json.load(f)
            data['gant_diagram'] = abs_output
            with open(self._source_file, 'w', encoding='utf-8') as f:
                json.dump(data, f, indent=4, ensure_ascii=False)
        except Exception:
            pass
    return abs_output
```

Listing 5.36: Metoda służąca do zapisu diagramu Gantta

Po utworzeniu wykresu, konieczne jest jego zapisanie w docelowym katalogu `/results`. Metoda `save()`, zaprezentowana na listingu 5.36, odpowiada za tę operację oraz za aktualizację pliku źródłowego harmonogramu (`*.JSON`) o ścieżkę do wygenerowanego wykresu. Zapewnia to, że interfejs użytkownika ma zawsze bezpośredni i automatyczny dostęp do najnowszej wizualizacji.

5.2. Implementacja warstwy aplikacji klienckiej — Java

Z uwagi na brak łatwych w obsłudze bibliotek do interfejsów graficznych dla tak złożonego projektu w języku Python, warstwa kliencka systemu została zaimplementowana w Javie z wykorzystaniem frameworka JavaFX. Wybór ten podyktowany był wcześniejszym doświadczeniem w tworzeniu aplikacji w tym środowisku.

Całość interfejsu została zrealizowana zgodnie z architektoniczną konwencją *Model View Controller* (MVC). Poniższy opis został zrealizowany z perspektywy użytkownika końcowego, odzwierciedlając sekwencję interakcji z aplikacją.

5.2.1. Interfejs użytkownika oraz wprowadzenie danych lub ich generacja

Po uruchomieniu aplikacji w pierwszej kolejności wykonywane jest sprawdzenie dostępności środowiska *Python*, które jest niezbędne do działania algorytmów obliczeniowych. Dostępność ta jest weryfikowana przy pomocy funkcji `checkPythonAvailability()`.

```
private boolean checkPythonAvailability() {
    try {
        ProcessBuilder pb = new ProcessBuilder("python", "--version"
    );
        pb.redirectErrorStream(true);
        Process process = pb.start();
        int exitCode = process.waitFor();
        if (exitCode == 0) {
            return true;
        }
        pb = new ProcessBuilder("python3", "--version");
        pb.redirectErrorStream(true);
        process = pb.start();
        exitCode = process.waitFor();
        return exitCode == 0;
    } catch (Exception e) {
        System.err.println("Błąd podczas sprawdzania Pythona: " + e.
    getMessage());
        return false;
    }
}
```

Listing 5.37: Metoda sprawdzająca czy *Python* jest zainstalowany

Listing 5.37 przedstawia metodę z klasy `SchedulerApplication`, która odpowiada za to sprawdzenie. W tym celu buduje proces systemowy (`ProcessBuilder`), który wykonuje polecenie w konsoli. Kluczowe jest to, że funkcja próbuje wykonać polecenie z użyciem standardowej nazwy `"python"`, a w przypadku niepowodzenia automatycznie stosuje mechanizm zastępczy, próbując wywołać `"python3"`. Proces uznaje się za zakończony sukcesem, gdy kod wyjścia (`exitCode`) jest równy zero. W zależności od rezultatu testu:

- Jeśli proces się powiedzie, aplikacja przechodzi do wyświetlania głównego widoku.
- W przypadku niepowodzenia (brak środowiska lub wystąpienie wyjątku `Exception`) wyświetlany jest błąd w konsoli, a aplikacja nie jest uruchamiana.

Główny widok aplikacji (klasa `MainMenuView`) charakteryzuje się minimalistyczną i intuicyjną strukturą. Służy on jako centralny punkt nawigacyjny, prezentując dostępne opcje i operacje do wykonania.

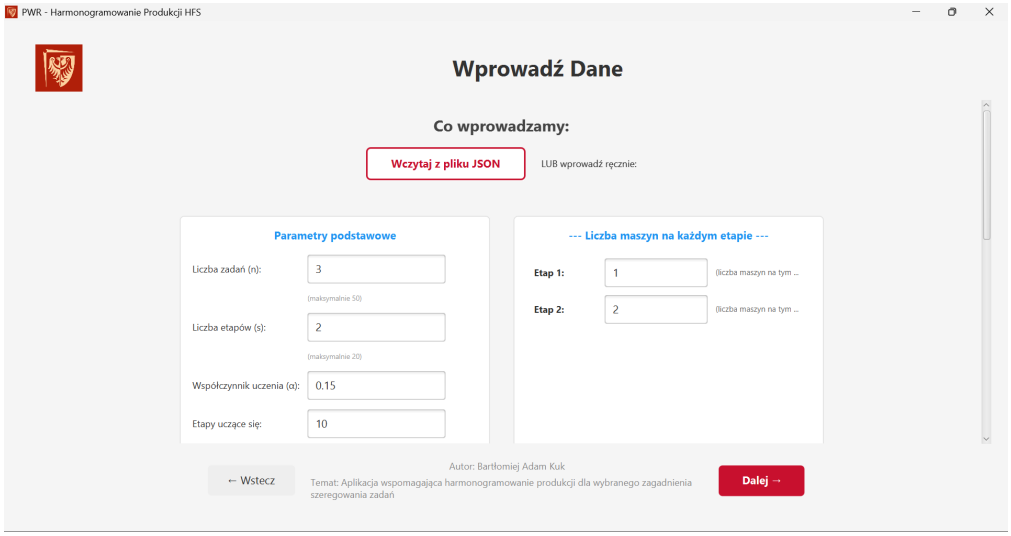
Rysunek 5.2: Widok głównego okna uruchomionej aplikacji

Na Rysunku 5.2 zaprezentowano główny widok aplikacji. Zgodnie z finalną implementacją, do wyboru są 3 opcje, zamiast 4 przewidzianych w pierwotnym szkielecie aplikacji (Sekcja 4.4). Decyzja ta wynikała z faktu, iż logiczne ścieżki dla opcji tworzenia nowego harmonogramu oraz generowania losowego harmonogramu były zbieżne (obie prowadziły do widoku wprowadzania danych), eliminując potrzebę utrzymywania dwóch osobnych akcji w interfejsie użytkownika.

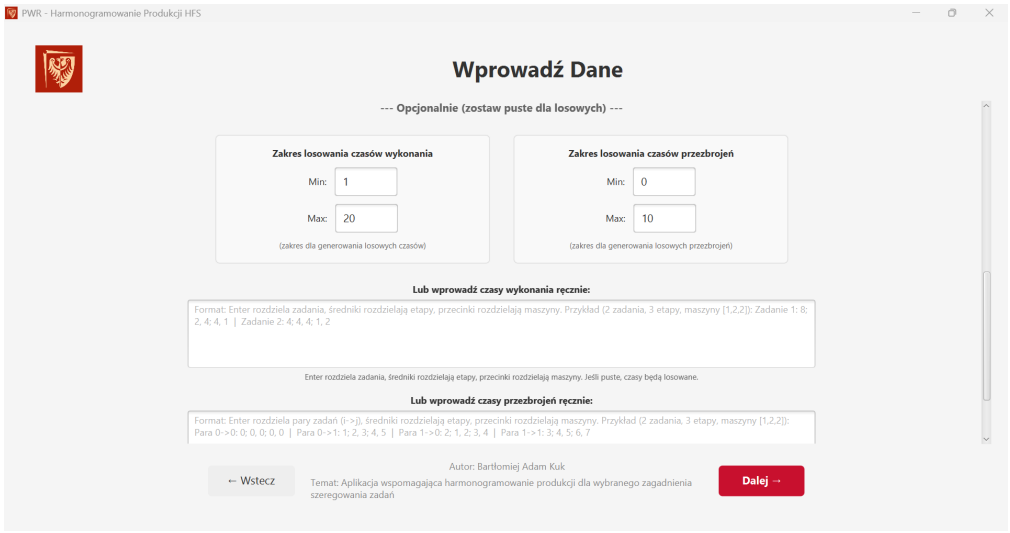
Opcje *Załaduj instancje z pliku* oraz *Wygeneruj wykres Gantta z istniejącego harmnonogramu*. Obie te opcje otwierają nowe okno systemowe, w którym należy dodać plik `*.JSON`.

Opcja 3 *Tworzenie lub generowanie harmnonogramu*, przekierowuje użytkownika do widoku `DataInputView`. Widok ten umożliwia wprowadzanie danych dla instancji, a poniżej znajduje się pełna lista parametrów niezbędnych do poprawnego działania hybrydowego przepływowego problemu szeregowania zadań: (1) liczba zadań, (2) ilość etapów,(3) ilość maszyn na każdym etapie, (4) czas przetwarzania na każdej maszynie dla każdego zadania, (5) wskazać który etap jest uczący się, (6) wartośc współczynnika uczenia, (7) tablice z czasami przezbrojeń na każde zadanie.

Poniżej, na rysunkach 5.3 oraz 5.4, przedstawiono interfejs użytkownika służący do wprowadzania parametrów problemu. W ostatecznej implementacji układ ten różni się od założeń projektowych z Sekcji 4.4. Pierwotne podejście zakładające sekwencyjne wprowadzanie danych mogłoby prowadzić do nadmiernej fragmentacji kodu i zwiększenia liczby niezależnych widoków. Z tego względu zdecydowano się na rozwiązanie, które prezentuje dane w logicznie podzielonych panelach w ramach jednego głównego okna.



Rysunek 5.3: Pierwsza część wprowadzania danych



Rysunek 5.4: Druga część wprowadzania danych

Za ustawienie takiego widoku odpowiadają metody klasy `DataInputView`: `createLayout`, `createHeader`, `createCenterPanel`, `createFormContainer`, `createLeftColumn`, `createRightColumn` oraz `createOptionalFieldsSection`. Do organizacji layoutu wykorzystywany jest kontener `BorderPane` z biblioteki JavaFX. Każda z nich jest rozbudowana i tworzy wszystkie kontenery w uporządkowany sposób.

Wymienione metody odpowiadają za hierarchiczną budowę widoku, gdzie każda z nich jest odpowiedzialna wyłącznie za stworzenie i ułożenie swojego bezpośredniego kontenera. Na przykład, metoda `createLayout()` wywołuje metody `createHeader()` , tworząc nagłówek aplikacji, `createCenterPanel()` , tworząc centralny panel z operacjami, oraz `createFooter()` do stworzenia stopki. Podział taki zapewnia pełną modularność kodu interfejsu użytkownika.

Metoda `createLayout()` umieszcza centralny panel widoku w kontenerze `ScrollPane`. Jest to niezbędne, ponieważ formularz do wprowadzania parametrów jest rozbudowany i wymaga przewijania, co zapewnia aplikacji wysoką użyteczność niezależnie od rozdzielczości ekranu użytkownika końcowego.

Wprowadzone parametry są poddawane wstępnej walidacji w celu zapewnienia spójności i poprawności danych wejściowych, jeszcze przed uruchomieniem silnika optymalizacyjnego. Mechanizm ten sprawdza krytyczne ograniczenia problemu (np. minimalna liczba maszyn, nieujemne czasy). W przypadku stwierdzenia błędu użytkownik otrzymuje natychmiastowe sprzężenie zwrotne w formie okna dialogowego (pop-up), które precyzyjnie informuje o niezgodności. Przykładowo, komunikat walidacyjny wymaga, aby współczynnik uczenia α należał do wymaganego zakresu $[0, 1)$.

Za wprowadzanie danych odpowiada moduł wizualizacji `DataInputView.java`, który odpowiada za prezentacje okna przedstawionego na rysunku 5.3 oraz 5.4.
Umożliwia ona również walidację danych wprowadzonych przez użytkownika.

Walidacja danych – widok wprowadzania danych odpowiada również za walidację danych. Sprawdza, czy każdy z parametrów spełnia następujące kryteria:

1. Czy liczba zadań n jest z zakresu $n \in [1, 50]$.

2. Czy liczba etapów S mieści się w zakresie $S \in [1, 20]$.

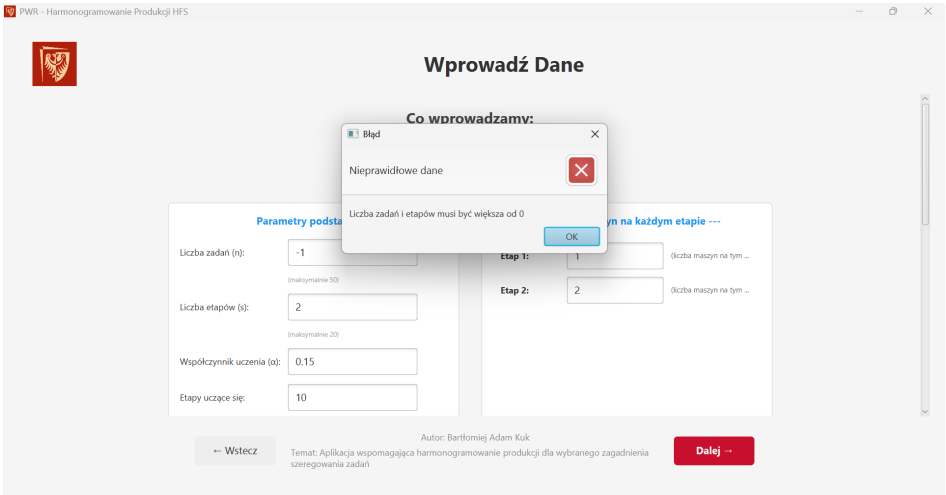
3. Czy liczba maszyn m_s na każdym etapie $s \in \{1, \ldots, S\}$ jest większa od 0.

4. Czy współczynnik uczenia α znajduje się w przedziale $\alpha \in [0, 1]$.

5. Czy długość wprowadzonego ciągu etapów uczących się (`learningStages`) jest **równa** całkowitej liczbie etapów S.

6. Czy przynajmniej jeden etap s posiada *dwie lub więcej maszyn równoległych* ($m_s \geq 2$), co jest fundamentalnym warunkiem dla HFP.

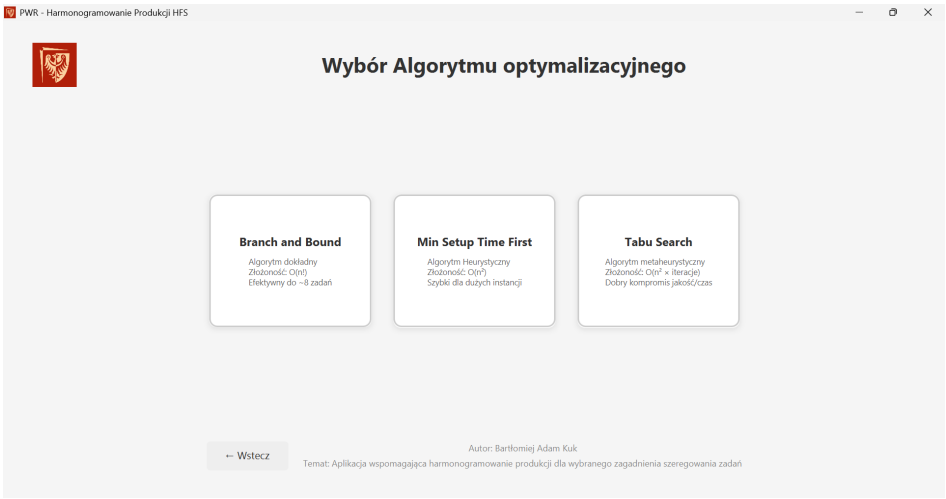
7. Czy format danych w polach opcjonalnych (czasy przetwarzania, czasy przezbrojeń) jest poprawny (walidacja `try-catch` przechwytująca błędy `NumberFormatException`).



Rysunek 5.5: Przykład wyskakującego okienka z informacją o błędzie

Na rysunku 5.5 przedstawia przykładowy błąd, który wyskakuje w nowym oknie przy użyciu funkcji `showError(...)`. jest ona uniwersalna i może być stosowana do każdego typu błędu z odpowiednią informacją zwrotną.

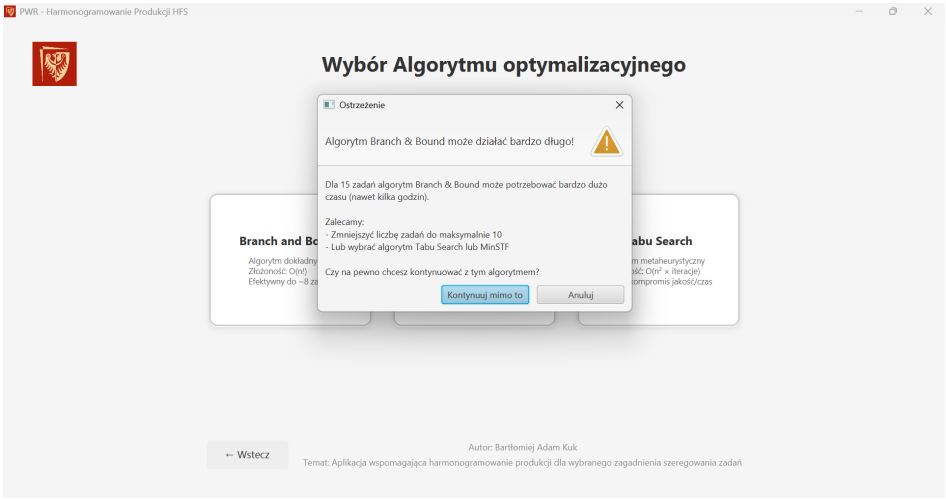
Aby łatwo operować na całości danych instancji problemu oraz zapewnić centralny punkt wymiany informacji, została utworzona klasa `ProblemInstance`. Klasa ta pełni rolę agregatora danych (modelu), enkapsulując wszystkie kluczowe parametry problemu HFS (liczbę zadań, etapów, czasy przetwarzania, współczynnik uczenia itp.). Dzięki temu cały stan problemu jest zawarty w jednym obiekcie. Kluczowe dla architektury aplikacji są metody `toJson` oraz `fromJson`. Umożliwiają one łatwą serializację i deserializację całego obiektu instancji zapisanego w formacie `*.JSON`. Po zdefiniowaniu parametrów instancji problemu można przejść do *wyboru algorytmu optymalizacji*. Widok ten jest obsługiwany w pliku `AlgorithmSelectionView`.



Rysunek 5.6: Widok wyboru algorytmu dla zdefiniowanego już problemu

Wybór algorytmu (Rysunek 5.6) polega na kliknięciu algorytmu, który chcemy uruchomić. Dodatkowo każdy guzik posiada swój opis, jaki jest to algorytm, jego złożoność oraz krótką informacje o algorytmie, np. *Branch and Bound, Algorytm dokładny, złożonośc* $O(n!)$ *Efektywny do ~8 zadań.* Pozwala to na wybór algorytmu nawet dla osoby, która nie jest specjalistą w tym temacie.

Dodatkowo dodano również ostrzeżenie, jeżeli ktoś spróbuje wykonać algorytm *Branch and Bound* dla harmonogramu z większą ilością zadań niż 10.



Rysunek 5.7: Przykład ostrzerzenia podczas próby wywołania algorytmu $B\&B$ dla dużej instancji

Za wyświetlenie takiego okna służy funkcja `handleAlgorithmSelection()`. Pozwala ona wybrać, czy użytkownik akceptuje ryzyko długiego oczekiwania. Po wyborze algorytmu interfejs użytkownika przechodzi do widoku, który przedstawia oczekiwanie na wyniki algorytmu optymalizacji. Realizowany jest w pliku `OptimizationProgressView`. Odpowiada wywołanie głównego skryptu Pythona (silnik optymalizacyjny), przekazując mu całą instancję problemu (`ProblemInstance`) oraz nazwę wybranego algorytmu. Szczegółowy mechanizm komunikacji między GUI a skryptem Pythona został opisany w Sekcji 5.2.2.

5.2.2. Integracja z procesem Pythonem

Budowa systemu w oparciu o dwa różne języki programowania (Java i Python) narzuciła konieczność rozwiązania kluczowego problemu komunikacji i synchronizacji danych między komponentami. W tym celu wykorzystano klasę `ProcessBuilder`. Rozwiązanie to tworzy nowy proces systemowy, który uruchamia aplikację napisaną w *Pythonie*.
Nie jest to rozwiązanie idealne, ponieważ zmusza użytkownika końcowego do posiadania dwóch środowisk uruchomieniowych języków programowania. Ma to swoje zalety, ponieważ odpowiednia implementacja silnika obliczeniowego umożliwia reużywalność dla innych problemów lub rozwiązań. Za obsługę zależności oraz wywołanie skryptu *Pythona* odpowiada `PythonService`. Serwis ten odpowiada za:

1. `runOptimization(...)` – uruchamia algorytm optymalizacji dla podanej instancji problemu. Metoda waliduje dane wejściowe, zapisuje je do pliku JSON, uruchamia proces Pythona z odpowiednimi parametrami, oczekuje na zakończenie (z limitem czasu 600 sekund) oraz wczytuje i zwraca wyniki harmonogramowania.

2. `isPythonAvailable()` – sprawdza, czy Python jest dostępny w systemie poprzez uruchomienie komendy `python -version` i weryfikację kodu wyjścia.

3. `getPythonVersion()` – pobiera i zwraca wersję zainstalowanego interpretera Pythona jako ciąg znaków.

4. `getResultsDir()` – zwraca ścieżkę do katalogu z wynikami obliczeń.

5. `getGanttChartPath(...)` – zwraca ścieżkę do ostatnio wygenerowanego wykresu Gantta dla danego algorytmu.

6. `generateGanttChart(...)` – generuje wykres Gantta dla istniejącego pliku z harmonogramem poprzez wywołanie skryptu Pythona z parametrem `-gantt` i zwraca ścieżkę do wygenerowanego obrazu.

7. `cleanResultsFolder()` – czyści folder `results` z tymczasowych plików przy zamknięciu aplikacji, usuwając wszystkie pliki wewnątrz katalogu.

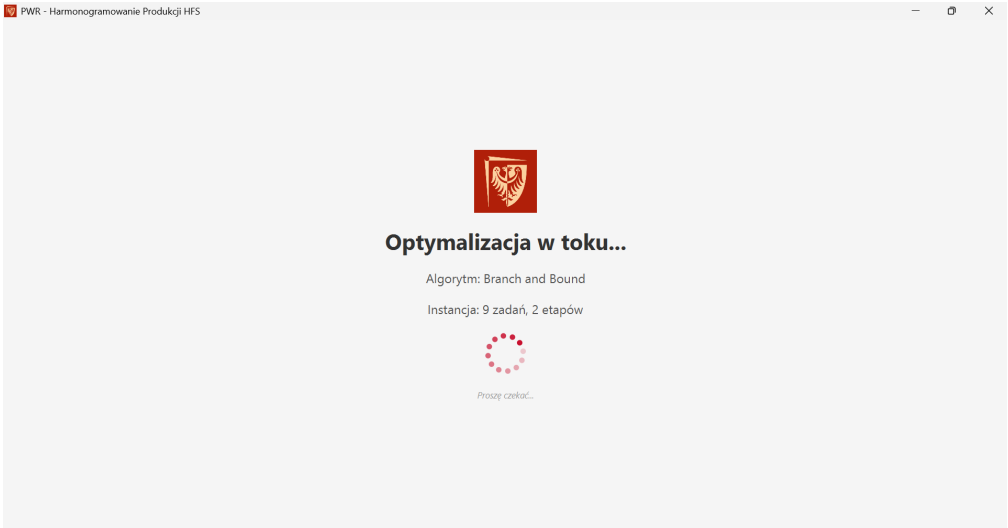
Klasa ta jest wykorzystywana podczas pokazywania widoku związanego z czekaniem na wyniki działania algorytmu. realizowana jest w klasie `OptimizationProgressView`. Kluczową metodą tej funkcji jest `startOptimization()`.

```
private void startOptimization() {
    new Thread(() -> {
        try {
            String pythonPath = findPythonProjectPath();
            PythonService pythonService = new PythonService(
    pythonPath);
            ScheduleResult result = pythonService.runOptimization(
    instance);
            Platform.runLater(() -> {
                ResultsView resultsView = new ResultsView(stage,
    result, instance.getAlgorithm(), instance);
                stage.getScene().setRoot(resultsView.getRoot());
            });
        } catch (Exception e) {
            e.printStackTrace();
            Platform.runLater(() -> showError());
        }
    }).start();
}
```

Listing 5.38: Wywołanie skryptu `Pythona` z poziomu interfejsu użytkownika

Listing 5.38 przedstawia kluczowy moment wywołania skryptu optymalizacyjnego. Metoda ta uruchamia skrypt w nowym wątku, w celu uniknięcia blokowania GUI, co umożliwia wyświetlenie np. ekranu ładowania. Następnie, po znalezieniu ścieżki projektu napisanego w *Pythonie* tworzony jest serwis, który uruchamia `ProcessBuilder` przy użyciu metody `pythonService.runOptimization`.

W przypadku gdy wystąpi jakikolwiek błąd (linie 11-14), zostanie wyświetlona wiadomość w oknie dialogowym, z nagłówkiem: *„Wystąpił błąd podczas wykonywania optymalizacji”* wraz z krótkim opisem: *„Możliwe przyczyny błędu: Uszkodzony plik wejściowy, Brak wymaganych bibliotek, Nieprawidłowa konfiguracja systemu”*.



Rysunek 5.8: Przykładowe okno przectawiające oczekiwanie na wyniki algorytmu

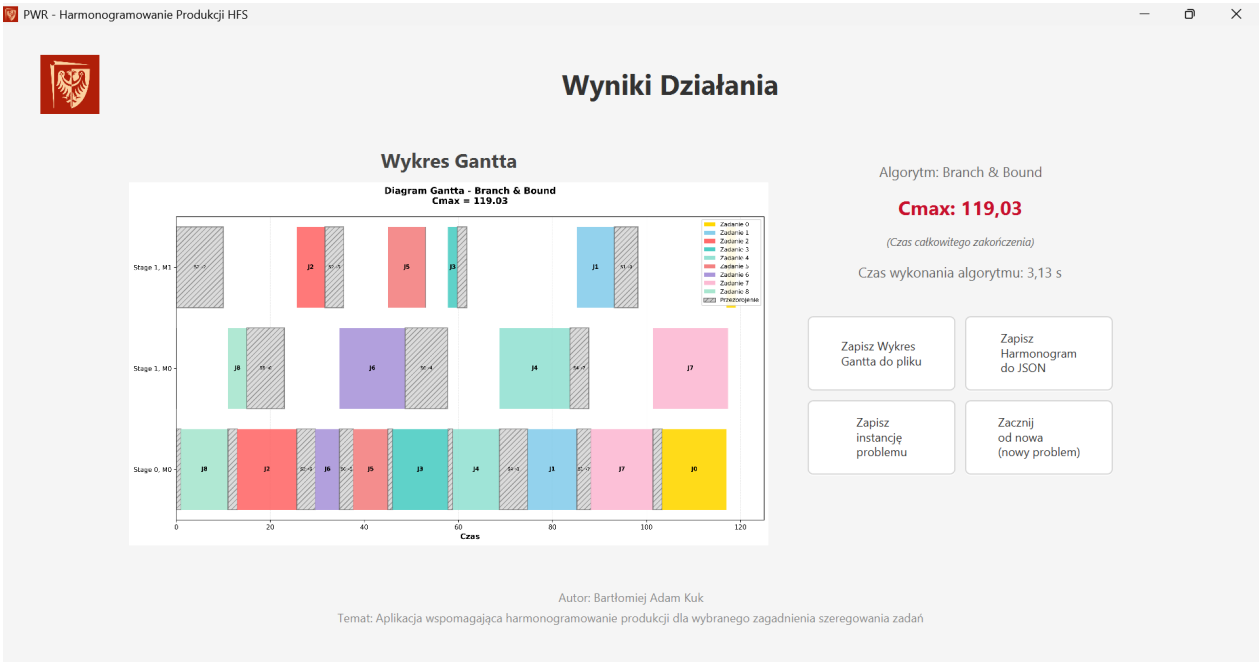
Po zakończeniu obliczeń (linia 7) przełączamy się z powrotem na główny wątek. Po tej operacji przechodzimy do widoku, który prezentuje wyniki algorytmu wraz z diagramem Gantta. Służy do tego klasa `ScheduleResult` oraz widok `ResultsView`, opisane w sekcji 5.2.3.

5.2.3. Wizualizacja wyników

Do prezentacji ostatecznych wyników służą dwie kluczowe klasy:

- `ScheduleResult` (Model Danych Wyjściowych) – Zawiera wszystkie informacje z pliku wynikowego zwracanego przez silnik obliczeniowy Pythona, w tym osiągnięty czas C_{max}, czas wykonania algorytmu oraz szczegółową listę operacji harmonogramu. Szczegółowy opis struktury danych znajduje się w sekcji 4.3.

- `ResultsView` (Widok Wyników) – Odpowiada za wyświetlenie tych danych oraz pełną obsługę interfejsu użytkownika w tym widoku, łącznie z panelem przycisków kontrolnych.



Rysunek 5.9: Przykładowe okno wynikowe aplikacji

Okno wynikowe (Rysunek 5.9) jest logicznie podzielone na dwie główne części: *diagram Gantta* oraz *część informacyjną*. Część informacyjna zawiera następujące metryki i elementy kontrolne:

- Osiągnięty czas harmonogramu ($\mathbf{C_{max}}$),
- Nazwa wykorzystanego *algorytmu*,
- *Czas wykonania* algorytmu.

Panel przycisków umożliwia interakcję z wynikiem, oferując następujące funkcje:

- **Zapisanie Wykresu Gantta** (eksport wizualizacji do pliku graficznego),
- **Zapisanie Harmonogramu** w formie pliku *.JSON* (eksport modelu `ScheduleResult`),
- **Zapisanie Instancji Problemowej** (`ProblemInstance`) w celu sprawdzenia innych algorytmów (reużywalność danych wejściowych),
- **Powrót do Głównego Ekranu** (`restart`).

Implementacja ta w pełni spełnia założenia interfejsu użytkownika określone w Sekcji 4.4.

5.3. Zarządzanie zależnościami i procedura uruchomieniowa

Podczas testowania aplikacji zauważyłem problem z częstym uruchamianiem i sprawdzaniem zależności. Z tego powodu stworzyłem plik wsadowy (`start.bat`), który automatyzuje cały proces uruchamiania, włączając w to walidację środowiska oraz instalację zależności.

Plik wsadowy wykonuje cztery główne, sekwencyjne kroki w celu przygotowania środowiska i uruchomienia aplikacji:

1. **Walidacja Java (`java`)**: Skrypt sprawdza obecność interpretera Javy (JDK 17+) w zmiennej systemowej `PATH` za pomocą polecenia `where java`. Jest to kluczowe, ponieważ interfejs graficzny jest napisany w Javie (JavaFX). W przypadku niepowodzenia wyświetla błąd i instrukcję pobrania.

2. **Walidacja Maven (`mvn`)**: Skrypt weryfikuje dostępność narzędzia Maven, które jest niezbędne do kompilacji i uruchomienia projektu JavaFX w środowisku deweloperskim.

3. **Walidacja Pythona (`python`)**: Skrypt sprawdza dostępność interpretera Pythona (wersja 3.8+), co jest warunkiem koniecznym do działania silnika obliczeniowego.

4. **Instalacja Zależności Python**: Skrypt przechodzi do katalogu `/Python` i używa polecenia `python -m pip install -r requirements.txt` do instalacji wszystkich wymaganych bibliotek. Ten krok zapewnia, że silnik obliczeniowy Pythona będzie posiadał wszystkie niezbędne pakiety.

Po pomyślnym sprawdzeniu i zainstalowaniu zależności plik wsadowy przechodzi do katalogu z projektem Java (`Demo_UI\javafx-ui`) i wywołuje polecenie:

```
call mvn javafx:run
```

Polecenie to powoduje kompilacje projektu wraz z załączeniem odpowiednich wtyczek.

6. Weryfikacja poprawności i napotkane problemy

W niniejszym rozdziale przedstawiono metodykę weryfikacji funkcjonalnej systemu poprzez testy jednostkowe, a także ocenę wydajności zaimplementowanych algorytmów na podstawie eksperymentów obliczeniowych oraz analizy kluczowych wyzwań technicznych napotkanych podczas implementacji.

6.1. Metodyka testowania i weryfikacja funkcjonalna

Do przeprowadzania testów oprogramowania wykorzystano bibliotekę `pytest`. Framework ten umożliwia realizację testów jednostkowych oraz tworzenie obiektów pozornych (mock-upów) dla poszczególnych funkcji. Poprawność działania oraz zgodność z wymaganiami zaimplementowanych modułów zweryfikowano poprzez wykorzystanie asercji.

Organizacja testów jednostkowych oparta została na hierarchicznym modelu frameworku `Pytest`, co zapewnia modularność, czytelność oraz efektywne zarządzanie przypadkami testowymi. Hierarchia ta, odwzorowująca strukturę testowanej klasy, składa się z trzech poziomów:

1. **Moduł Testowy (Batch/Zbiór Testów):**
   - Reprezentowany przez pojedynczy plik `.py` (np. `test_job.py`).
   - Stanowi zbiór wszystkich testów dedykowanych jednej klasie produkcyjnej.
2. **Test Suite (Zestaw Testów):**
   - Implementowany jako **klasa** (np. `TestJobInitialization`).
   - Grupuje logicznie powiązane przypadki testowe, koncentrując się na konkretnej funkcjonalności lub metodzie klasy (np. na inicjalizacji obiektu `Job` lub na metodzie `get_setup_time`).
3. **Test Case (Przypadek Testowy):**
   - Implementowany jako **pojedyncza metoda** w klasie `Test Suite` (np. `test_job_creation_with_valid_data`).
   - Jest to najmniejsza jednostka testująca, weryfikująca jeden, ściśle określony warunek lub ścieżkę wykonania kodu.

```python
def test_get_self_setup_time_with_none(self, sample_job):
    """Test pobierania czasu self-setup (prev_job=None)."""
     setup_time = sample_job.get_setup_time(prev_job=None, stage=0,
   machine=0)
     assert setup_time == 1.0
```

Listing 6.1: Przykładowy test case klasy `Job`

Listing 6.1 przedstawia przykładowy test case, z wykorzystaniem przygotowanych danych testowych. Sprawdza, jaki jest czas przezbrojenia na zadania samego na siebie na etapie zerowym na maszynie zerowej.

Table 6.1: Zestawienie Wyników Testów Jednostkowych Klasy `Job`

Test / Przypadek	**Ocz.**	**Otrz.**
TEST SUITE: Inicjalizacja		
test_job_creation_with_valid_data	PASSED	**PASSED**
test_job_creation_with_custom_name	PASSED	**PASSED**
test_job_initialization_arrays	PASSED	**PASSED**
TEST SUITE: Czas Przetwarzania		
test_get_processing_time_scalar	PASSED	**PASSED**
test_get_processing_time_list_format	PASSED	**PASSED**
test_get_processing_time_different_machines	PASSED	**PASSED**
test_get_processing_time_different_stages	PASSED	**PASSED**
TEST SUITE: Czas Setupu		
test_get_self_setup_time_with_none	PASSED	**PASSED**
test_get_self_setup_time_with_minus_one	PASSED	**PASSED**
test_get_setup_time_from_different_job	PASSED	**PASSED**
test_get_setup_time_list_format	PASSED	**PASSED**
test_get_setup_time_different_machines	PASSED	**PASSED**
TEST SUITE: Ustawienie Czasów Etapu		
test_set_stage_times_valid	PASSED	**PASSED**
test_set_stage_times_multiple_stages	PASSED	**PASSED**
test_set_stage_times_invalid_proc_before_setup	PASSED	**PASSED**
test_set_stage_times_invalid_end_before_start	PASSED	**PASSED**
test_set_stage_times_updates_total_time	PASSED	**PASSED**
test_set_stage_times_with_delay	PASSED	**PASSED**

Tabela 6.1 przedstawia wyniki testów dla klasy `Job`, sprawdzając przy tym: poprawność inicjalizacji obiektu, dokładność pobierania i obliczania czasów przetwarzania w różnych konfiguracjach maszyn i etapów, poprawność mechanizmu pobierania czasów przezbrojenia, walidację logiki sekwencjonowania czasów etapu, w tym poprawną aktualizację całkowitego czasu realizacji oraz kontrolę warunków brzegowych.

Table 6.2: Podsumowanie wyników testów jednostkowych (`pytest`) z podziałem na batche.

Batch (Moduł Logiczny)	**Testów**	**Zdanych**	**Niezdanych**	**% Sukcesu**
`TEST_JOB.PY`	18	18	0	100.00%
`TEST_MACHINE.PY`	27	27	0	100.00%
`TEST_SCHEDULE.PY`	34	34	0	100.00%
`TEST_GREEDY.PY`	22	22	0	100.00%
`TEST_TABU_SEARCH.PY`	36	36	0	100.00%
`TEST_BRANCH_AND_BOUND.PY`	18	18	0	100.00%
SUMA GLOBALNA	**155**	**155**	**0**	**100.00%**

Tabela 6.2 przedstawia podsumowanie testów jednostkowych, potwierdzając pełną weryfikację funkcjonalną poszczególnych modułów systemu. Ukazuje ona, że wszystkie 155 przypadków testowych przeszło pomyślnie, co świadczy o stabilności oraz zgodności zdefiniowanych funkcjonalności z oczekiwanymi wynikami.

Pokrycie testami jednostkowymi wynosi 94% dla kodu odpowiedzialnego za algorytmy optymalizacji (Branch and Bound, Greedy, Tabu Search) oraz klas kluczowych (Job, Machine, Schedule).

6.2. Eksperymenty obliczeniowe i ocena wydajności

Wydajność zaimplementowanych algorytmów została zweryfikowana w serii eksperymentów obliczeniowych, koncentrujących się na analizie czasu działania w zależności od wielkości instancji problemu.

Badania zostały podzielone na dwa zestawy:

1. Badanie czasu działania w zależności od liczby zadań
2. Badanie czasu działania w zależności od liczby etapów

Badania zostały wykonane na komputerze o następującej konfiguracji:
procesor Intel Core i5-8350U, 16 GB pamięci RAM, system operacyjny – `Windows` oraz wersja pythona 3.13.4. Każda z instancji testowych została uruchomiona 100 razy w celu uśrednienia wyników i zminimalizowania wpływu szumów systemowych na czas wykonania.

Wszystkie generowane instancje losowe charakteryzowały się następującymi stałymi parametrami:

- Ostatni etap jest uczący z współczynnikiem $\alpha = 0,3$.
- Zakres czasów przetwarzania jest losowany z przedziału $\langle 10, 20 \rangle$.
- Zakres czasów przezbrojeń jest losowany z przedziału $\langle 5, 10 \rangle$.
- Konfiguracja maszyn: Wszystkie etapy są równoległe z 5 maszynami.

Badanie czasu działania w zależności od liczby zadań (1)

Celem tego zestawu było przeanalizowanie wpływu wzrostu liczby zadań (n) na czas działania algorytmów. Zastosowano następujące parametry instancji:

- Liczba etapów (S): 5
- Liczba zadań (n): Zmienna w zbiorze $2, 4, 8, 16, 32$.

Badanie czasu działania w zależności od liczby etapów (2)

Celem tego zestawu było przeanalizowanie wpływu ilości etapów (S) na czas działania algorytmów. Przyjęto następujące parametry instancji:

- Liczba zadań (n): 10
- Liczba etapów (S): Zmienna w zbiorze $2, 4, 8, 16, 32$.

Wyniki badań

Niniejsza sekcja przedstawia szczegółowe wyniki eksperymentów obliczeniowych, które miały na celu weryfikację złożoności czasowej zaimplementowanych algorytmów.

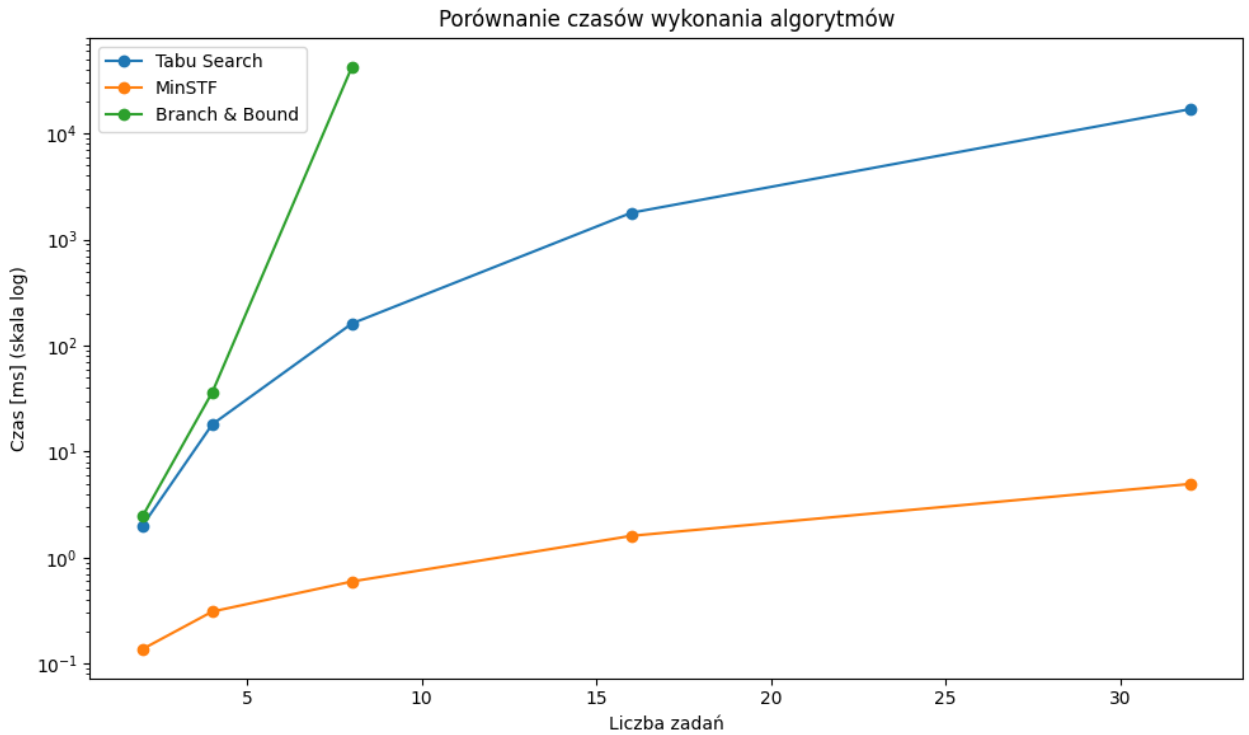
Badanie czasu działania w zależności od liczby zadań

Tabela 6.3 przedstawia średni czas wykonania algorytmów dla 100 różnych instancji. Przedstawia ona, iż średni czas wykonania jest zgodny z teoretycznym, czyli:

- `B&B` – $O(n!)$
- `MinSTF` – $O(n^2)$
- `TabuSearch` – $O(kn^2)$, gdzie k jest liczbą iteracji

Table 6.3: Skalowalność czasowa algorytmów dla różnej liczby zadań.

Liczba zadań (N)	**B&B [ms]**	**Tabu [ms]**	**MinSTF [ms]**
2	2.4457	1.9995	0.1371
4	36.2494	18.0677	0.3100
8	42516.9540	161.4856	0.5959
16	–	1789.0624	1.6048
32	–	16978.2789	4.9549



Rysunek 6.1: Porówanie czasów działania algorytmów

Na Wykresie 6.1 można zaobserwować, że czas działania algorytmu MinSTF jest znacznie krótszy niż Tabu Search. Choć oba algorytmy mają złożoność wielomianową, *Tabu Search* charakteryzuje się znacznie wyższą stałą i wykładnikiem złożoności ze względu na konieczność wykonania dużej i stałej liczby iteracji przeszukiwania przestrzeni sąsiedztwa. Mimo tego większego nakładu obliczeniowego, jak przedstawia Tabela 6.4, różnica jakości rozwiązania względem optimum (znalezionego przez B&B) jest znacząca i uzasadniona.

Table 6.4: Porównanie Jakości Rozwiązań (Średnia $C_{\max}$) względem Optimum Globalnego na 100 losowych instancjach

Rozmiar (N)	**B&B** ($\bar{C}_{\max}$)	**MinSTF** ($\bar{C}_{\max}$)	**Gap MinSTF** ($\bar{\Delta}$)	**Tabu Search** ($\bar{C}_{\max}$)	**Gap Tabu Search** ($\bar{\Delta}$)
$N = 2$	67.87	68.24	+0.5473%	67.87	+0.0000%
$N = 4$	71.06	76.10	+7.0929%	71.06	+0.0000%
$N = 8$	85.01	96.85	+13.9273%	85.96	+1.1177%

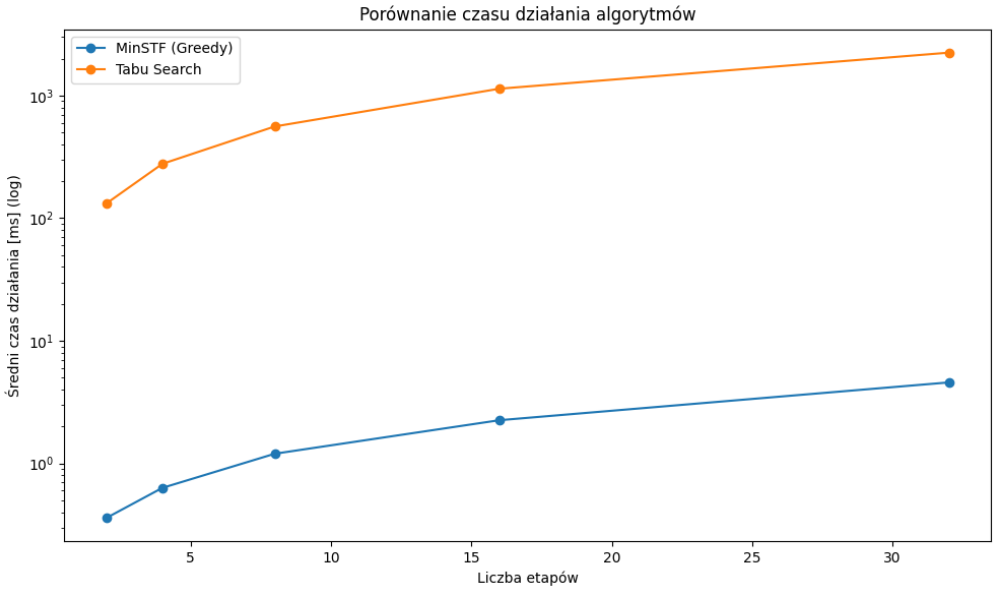
Przeprowadzone badania jakościowe, bazujące na *uśrednionych wynikach z 100 losowo generowanych instancji* dla każdej konfiguracji, wskazują na dużą przewagę algorytmu *Tabu Search*. Algorytm ten osiągnął optimum w 100% przypadków dla mniejszych instancji ($N = 2$ i $N = 4$), a dla całego zestawu badawczego ($N = 2, 4, 8$) średnio różnił się od optimum globalnego o zaledwie +0.3726%.

Kontrastuje to z heurystyką MinSTF, dla której średnia różnica od optimum dla wszystkich instancji wynosiła aż +7.1892%, potwierdzając, że proste metody szybko tracą efektywność w złożonym środowisku HFS-SDST. W związku z tym, *Tabu Search jest kluczowym wyborem w aplikacji*, gdy priorytetem jest uzyskanie wysokiej jakości rozwiązania.

Badanie czasu działania w zależności od liczby etapów

Table 6.5: Wpływ liczby etapów na czas wykonania algorytmów dla stałej ilości zadań.

Liczba Etapów	MinSTF (Greedy) [ms]	Tabu Search [ms]
2	0.35910	131.54020
4	0.63277	277.37090
8	1.20085	559.46216
16	2.25371	1136.54021
32	4.58083	2235.97922



Rysunek 6.2: Porówanie czasów działania algorytmów dla zmiennej ilości etapów przy stałej ilości zadań

Analiza tabeli 6.5 potwierdza złożoność $O(S)$ (liniową względem liczby etapów). Przy podwojeniu liczby etapów, np. z 8 na 16, czas wykonania obu algorytmów wzrasta wprost proporcjonalnie: dla MinSTF: Czas wzrasta ≈ 1.88 razy, natomiast dla Tabu Search: Czas wzrasta ≈ 2.03 razy.

Taki liniowy wzrost sugeruje, że liczba etapów ma liniowy wpływ na złożoność algorytmów, a co za tym idzie, nie stanowi wąskiego gardła dla skalowalności algorytmów w hybrydowym problemie przepływowym. Głównym czynnikiem limitującym, zgodnie z Badaniem 1, pozostaje liczba zadań.

6.3. Analiza napotkanych problemów

Podczas realizacji implementacji napotkano następujące problemy:

1. **Problem z komunikacją Python → Java** – Polegał on na tym, że interfejs użytkownika nie miał wiarygodnych informacji o kodzie wyjścia procesu oraz o tym, co zostało przesłane do standardowego strumienia błędu (`stderr`).
   Rozwiązanie: podczas implementacji skryptu w oknie błędu wyświetlał się natywny kod wyjścia zwrócony przez interpreter Pythona, co ułatwiło diagnostykę. W obecnej wersji treść tego błędu została zmieniona na bardziej przyjazną użytkownikom.

2. **Generowanie diagramów Gantta** — Złożoność problemu (Hybrydowy Problem Przepływowy z czasami przezbrojeń zależnymi od sekwencji oraz efektem uczenia) uniemożliwiała efektywne wykorzystanie konwencjonalnych metod generowania wykresów z biblioteki `matplotlib`. Problem polegał na konieczności wizualnego odróżnienia czasu przetwarzania od czasu przezbrojenia (SDST) oraz na poprawnej reprezentacji *różnej liczby maszyn* na poszczególnych etapach.
   Rozwiązanie: Opracowano dedykowany, niestandardowy skrypt wizualizacyjny. Skrypt ten wykorzystuje bibliotekę `matplotlib`, ale implementuje zaawansowaną logikę rysowania. Skrypt dokonuje iteracji po operacjach (włączając w to bloki czasu operacji oraz przezbrojeń) z obiektu `ScheduleResult`. Kluczowe było zastosowanie niestandardowego mapowania kolorów i etykiet, co pozwala na wizualne odróżnienie czasu przetwarzania zadania od czasu trwania przezbrojenia oraz zapewnia poprawną skalowalność osi Y, reprezentującej maszyny w poszczególnych etapach produkcji.

3. **Obsługa czasów przezbrojeń** – Złożoność czasów przezbrojeń zależnych od kolejności sprawiała problemy z ich poprawnym przetwarzaniem. W przeciwieństwie do stałych czasów przezbrojeń, SDST wymagają, aby algorytm optymalizacyjny na każdym etapie dynamicznie obliczał i wstawiał czas przezbrojenia maszyny. Wartość tego czasu zależy od typu zadania, które zostało wcześniej przetworzone na danej maszynie.
   Rozwiązanie: Moduł obliczeniowy w został zaprojektowany tak, aby przechowywać dane czasów przezbrojeń w formie macierzy w ramach struktury opisującej instancję problemu, a algorytmy optymalizacyjne zostały zmodyfikowane tak, aby na etapie planowania operacji na maszynie uwzględnić czasy SDST. Następuje to poprzez:

   - Identyfikacja ostatniego ukończonego zadania na danej maszynie.
   - Pobranie odpowiedniego czasu przezbrojenia z macierzy SDST w zależności od sekwencji zadań.
   - Wstawienie i zarejestrowanie czasu przezbrojenia jako osobnej operacji w modelu wynikowym `ScheduleResult`.

   Takie podejście zapewniło spójność w obliczeniach harmonogramu i umożliwiło poprawne wizualne odróżnienie czasu przetwarzania od czasu przezbrojenia w Wykresie Gantta (patrz: 5.2.3).

7. Zakończenie

W niniejszej pracy zrealizowano cel polegający na stworzeniu aplikacji umożliwiającej realizację hybrydowego problemu szeregowania zadań (HFS-SDST), z uwzględnieniem przezbrojeń zależnych od kolejności oraz efektu uczenia się. Osiągnięcie to zostało zrealizowane poprzez implementację i ocenę wydajności algorytmu dokładnego Branch & Bound oraz dwóch heurystyk: algorytmu MinSTF i zaawansowanej metaheurystyki Tabu Search.

Aplikacja, poza implementacją rdzenia obliczeniowego, umożliwia użytkownikowi w przystępny sposób podanie parametrów problemu, zapis instancji, wybór algorytmu optymalizacyjnego oraz wizualną prezentację wyników (w tym generowanie wykresów Gantta).

Najważniejszymi elementami wykonanymi w ramach pracy było:

- Stworzenie interfejsu użytkownika oraz komunikacja z skryptem obliczeniowym,
- Wdrożenie logiki obliczania funkcji celu ($C_{\max}$), uwzględniającej jednocześnie trzy założone czynniki: czas przetwarzania, czas przezbrojenia zależny od kolejności oraz efekt uczenia się,
- Opracowanie dedykowanego skryptu wizualizacyjnego w Pythonie do generowania diagramów Gantta, co wymagało niestandardowej logiki mapowania czasów przetwarzania i przezbrojeń na maszynach,
- Implementacja algorytmów B&B, MinSTF i Tabu Search i przygotowanie ich do eksperymentalnej weryfikacji.

Eksperymenty obliczeniowe wykazały, iż ilość etapów ma liniowy wpływ na czas działania algorytmów, ważniejszą rzeczą jest ilość zadań. Algorytm B&B nie jest odpowiedni dla dużych instancji ze względu na swoją wykładniczą złożoność $O(n!)$, pomimo że zawsze daje optymalne rozwiązanie. Kompromisem może być zastosowanie algorytmów heurystycznych lub metaheurystycznych. W przypadku tej pracy zrealizowane *Tabu Search* osiągneło wyniki bliskie optimum – gorsze o 0.3726%. Natomiast reguła priorytetowa *MinSTF*, pomimo swojego błyskawicznego czasu działania, daje relatywnie dobre rozwiązanie – gorsze o 7.1892% od optimum.

Projekt można dalej rozwijać w kilku kluczowych obszarach, obejmujących zarówno warstwę obliczeniową, jak i ogólną funkcjonalność aplikacji. W pierwszej kolejności warto poszerzyć zestaw dostępnych algorytmów o kolejne metaheurystyki, takie jak Symulowane Wyżarzanie (SA) czy Algorytm Genetyczny (GA). Pozwoliłoby to lepiej radzić sobie z dużymi instancjami problemu oraz osiągać rozwiązania o jeszcze mniejszym błędzie. Dodatkowo, warto rozważyć zastosowanie alternatywnych struktur sąsiedztwa w ramach Tabu Search, co może poprawić jakość uzyskiwanych wyników i umożliwić bardziej efektywne przeszukiwanie przestrzeni rozwiązań. Model samego problemu również można rozszerzać, m.in. o optymalizację wielokryterialną, czasy dostawy czy elementy związane z niezawodnością i awariami maszyn, aby zwiększyć realizm symulacji i jej potencjał praktyczny.

W kontekście rozwoju aplikacji warto skupić się na funkcjach, które zwiększą wygodę użytkownika końcowego — takich jak możliwość importowania danych wejściowych z formatów `*.csv` lub `*.xlsx`. Z punktu widzenia architektury oprogramowania istotnym kierunkiem może być także uproszczenie komunikacji między modułami. Zamiast wykorzystywać zewnętrzne procesy czy odrębne środowiska, można rozważyć przeniesienie zarówno warstwy obliczeniowej, jak i interfejsu na jedną spójną technologię, np. pełną implementację w C++. Pozwoliłoby to na wyeliminowanie narzutów wynikających z integracji dwóch języków oraz zwiększyłoby wydajność, przewidywalność i łatwość utrzymania całego rozwiązania.

Bibliografia

[1] H. Harbaoui and S. Khalfallah. Tabu-search optimization approach for no-wait hybrid flow-shop scheduling with dedicated machines. *Procedia Computer Science*, 176:706–712, 2020.

[2] X. Hu, L. Li, and X. Zhao. A branch-and-bound algorithm for project scheduling. *Mathematical Problems in Engineering*, 2015.

[3] A. Kurt. Integrating sequence-dependent setup times and blocking in hybrid flow shop scheduling to minimize total tardiness. *International Journal of Industrial Engineering Computations*, 16(1):147–158, 2025.

[4] C. Liu and W. Zhang. A branch and bound algorithm for the robust parallel machine scheduling problem with sequence-dependent setup time. *arXiv preprint arXiv:1712.00756*, 2017.

[5] L. Pugliese, M. Romano, Y. Lin, W. Zhu, and Y. Tan. Hybrid flow shop scheduling through reinforcement learning: A systematic literature review. *ACM Computing Surveys*, 57(3):1–34, 2025.

[6] I. Ribas, R. Companys, and X. Tort-Martorell. An iterated greedy algorithm for the parallel blocking flow shop scheduling problem with sequence-dependent setup times. *Expert Systems with Applications*, 170:114511, 2021.

[7] World Manufacturing Foundation. World manufacturing report 2024: Manufacturing outlook 2030, 2024. Ogólnodostępny raport dotyczący globalnych trendów w produkcji.

[8] W. Yang, S. Li, G. Luo, H. Li, and X. Wen. A real-time human–machine–logistics collaborative scheduling method considering workers' learning and forgetting effects. *Applied System Innovation*, 8(2):40, 2025.

[9] W. Yong. Heuristics for the hybrid flow shop scheduling problem with energy consumption and sequence-dependent setup times. *Mathematical Problems in Engineering*, 2022:1–13, 2022.

[10] Z. Zhang, Z. Shao, W. Shao, J. Chen, and D. Pi. Mrlm: A meta-reinforcement learning-based metaheuristic for hybrid flow-shop scheduling problem with learning and forgetting effects. *Advanced Engineering Informatics*, 60:102445, 2024.

Spis rysunków

4.1 Diagram komunikacji pomiędzy JavaFX a modułem optymalizacji w Pythonie 16
4.2 Widok okna startowego po uruchomieniu aplikacji. 23
4.3 Widok ekranu wyboru algorytmu optymalizacyjnego. 24
4.4 Widok ekranu wprowadzania parametrów instancji problemu. 24
4.5 Widok ekranu z wynikami działania algorytmu. 24

5.1 Przykładowy wykres Gantta dla hybrydowego przepływowego problemu szeregowania zadań 54
5.2 Widok głównego okna uruchomionej aplikacji 56
5.3 Pierwsza część wprowadzania danych 57
5.4 Druga część wprowadzania danych 57
5.5 Przykład wyskakującego okienka z informacją o błędzie 59
5.6 Widok wyboru algorytmu dla zdefiniowanego już problemu 59
5.7 Przykład ostrzerzenia podczas próby wywołania algorytmu $B\&B$ dla dużej instancji 60
5.8 Przykładowe okno przectawiające oczekiwanie na wyniki algorytmu 62
5.9 Przykładowe okno wynikowe aplikacji 63

6.1 Porówanie czasów działania algorytmów 69
6.2 Porówanie czasów działania algorytmów dla zmiennej ilości etapów przy stałej ilości zadań 70

Spis tabel

6.1 Zestawienie Wyników Testów Jednostkowych Klasy `Job` 66
6.2 Podsumowanie wyników testów jednostkowych (`pytest`) z podziałem na batche. 67
6.3 Skalowalność czasowa algorytmów dla różnej liczby zadań. 68
6.4 Porównanie Jakości Rozwiązań (Średnia $C_{\max}$) względem Optimum Globalnego na 100 losowych instancjach 69

6.5 Wpływ liczby etapów na czas wykonania algorytmów dla stałej ilości zadań. . 70